Université Internationale de Casablanca

UNIVERSITÉ RECONNUE PAR L'ÉTAT

CAIDELEC TRAVAUX -EQUIPEMENTS ELECTRIQUES

PROJET DE FIN D'ÉTUDES

Mémoire présenté en vue de l'obtention du diplôme de licence en Génie des Systèmes d'Information

Sous le thème

Système de gestion de la planification des interventions quotidiennes et des frais de déplacement

Réalisé par : Yassir Kemmissa

Encadrante académique : Mme Sanae EL OUKKAL

Encadrant professionnel : M. Boubker SETTAR

Soutenu le 18/09/2026 devant les membres du jury :

M. EL KASSIMI Mohamed	Examinateur Rapporteur
Mme EL OUKKAL Sanae	président
Mme ADIB Jihad	Examinateur Rapporteur

Année universitaire 2025–2026

Dédicace

Je dédie ce travail à mes parents, qui m'ont accompagné avec patience et confiance tout au long de mon parcours. Leur soutien constant, leurs sacrifices et leurs encouragements m'ont permis d'avancer, y compris dans les périodes de doute et de fatigue. Ils m'ont transmis le sens de l'effort, de la responsabilité et de la persévérance. Ce projet représente une étape importante de mes études, mais il porte surtout la trace de tout ce qu'ils ont fait pour me donner les moyens de réussir.

Je le dédie également à ma famille, pour sa présence, son affection et l'intérêt qu'elle a toujours accordé à mes études. Les conseils, les paroles rassurantes et les gestes simples reçus au fil des années ont contribué à maintenir ma motivation et à donner du sens au travail accompli.

À mes proches et à mes amis, j'adresse une pensée reconnaissante pour leur compréhension et leur soutien. Ils ont su respecter les contraintes liées à ce projet, m'encourager lorsque la charge de travail devenait importante et partager avec moi les moments de satisfaction qui ont accompagné son avancement.

Je dédie enfin ce rapport à toutes les personnes qui ont participé, directement ou indirectement, à ma formation académique et personnelle. Chaque échange, chaque conseil et chaque expérience a contribué à construire le parcours qui m'a conduit jusqu'à ce projet de fin d'études.

À toutes ces personnes, avec toute ma gratitude.

Remerciements

Je tiens tout d'abord à remercier Mme Sanae EL OUKKAL, mon encadrante académique, pour son accompagnement, ses remarques méthodologiques et ses exigences concernant la qualité du rapport et de la présentation. Ses retours m'ont conduit à reprendre la structure du document, à justifier les choix techniques et à améliorer la cohérence entre l'analyse, la conception et la réalisation.

J'adresse également mes sincères remerciements à M. Boubker SETTAR, mon encadrant professionnel au sein de CAIDELEC, pour sa disponibilité et pour les informations concrètes qu'il m'a fournies sur l'organisation des interventions, l'affectation des équipes et le contrôle des frais de déplacement.

Je remercie l'ensemble des collaborateurs de CAIDELEC qui ont pris le temps de répondre à mes questions et de m'aider à comprendre le fonctionnement quotidien de l'entreprise. Leurs explications ont été essentielles pour transformer un besoin opérationnel en fonctionnalités testables.

Enfin, je remercie l'Université Internationale de Casablanca ainsi que les enseignants de la filière Gestion des Systèmes d'Information pour les connaissances acquises durant ma formation et pour les méthodes de travail mobilisées dans ce projet de fin d'études.

Résumé

Dans le cadre de ce projet de fin d'études réalisé au sein de CAIDELEC, une application web de gestion des interventions quotidiennes et des frais de déplacement a été conçue et développée. Le besoin initial concernait la centralisation du planning des équipes de terrain, la réduction des erreurs d'affectation et l'amélioration de la traçabilité des dépenses associées aux chantiers.

La solution repose sur une architecture découplée composée d'une application monopage développée avec React et Vite, d'une API REST réalisée avec FastAPI, d'une couche d'accès aux données SQLAlchemy et d'une base locale SQLite. L'authentification utilise des jetons JWT et le contrôle d'accès distingue deux profils : l'administrateur et l'employé. L'administrateur gère les employés, planifie les interventions, consulte le calendrier, contrôle les frais et valide les déclarations. L'employé consulte ses missions, déclare leur réalisation et saisit ses dépenses avec un justificatif photographique.

Le système propose également des tableaux de bord adaptés au rôle, un suivi par statuts et l'export des affectations journalières au format Excel. La solution obtenue apporte une vision centralisée des opérations et constitue une base évolutive pour l'ajout de notifications, de rapports PDF, d'un historique d'audit et d'un déploiement de production renforcé.

Mots-clés : planification, interventions, frais de déplacement, React, FastAPI, SQLAlchemy, SQLite, JWT, système d'information.

Abstract

This graduation project was carried out at CAIDELEC and concerns the design and development of a web application for managing daily field interventions and the related travel expenses. The initial need was to centralize team planning, reduce assignment errors and make expense declarations easier to trace and verify.

The delivered solution follows a decoupled architecture. Its user interface is built with React and Vite, while the REST API is developed with FastAPI. SQLAlchemy provides the data access layer and SQLite is used for the stage version. Authentication is based on JWT access tokens and separates two profiles. The administrator manages employees, schedules interventions, consults the calendar, reviews expenses and validates declarations. The employee consults assigned missions, reports their completion and submits expenses together with a photographic receipt.

The application also provides role-based dashboards, status tracking and a daily Excel export of assignments. The resulting system centralizes the main operational data and provides a practical foundation for future improvements such as notifications, audit history, PDF reports, PostgreSQL migration and a production-ready deployment process.

Keywords: planning, field interventions, travel expenses, React, FastAPI, SQLAlchemy, SQLite, JWT, information system.

Table des matières

PROJET DE FIN D'ÉTUDES.....
Dédicace i
Remerciements ii
Résumé iii
Abstract iv
Table des matières v
Liste des figures..... vii
Liste des tableaux viii
Liste des acronymes ix
Introduction générale..... x
Introduction 1
Cadre général du projet 4
Étude préalable et cahier des charges 8
Conduite du projet 14
Analyse et spécification des besoins 20
Conception et architecture..... 30
Réalisation de l'application 40
Validation et résultats 51
Conclusion générale et perspectives..... 58
Références bibliographiques 59
Annexe A — Protocole de démonstration 60
Annexe B — Traçabilité de l'évaluation 61
Annexe C — Exécution de la version livrée 62
Annexe D Corrections et fonctionnalite ajoutee..... 63
Diagramme de cas d'utilisation corrige 64
Diagramme de classes corrige 65
Diagramme de sequence corrige 66
Diagramme d'activite corrige 67

Liste des figures

Figure 4.1 — Diagramme des cas d'utilisation du système CAIDELEC 23
Figure 5.1 — Architecture logique de l'application 30
Figure 5.3 — Cycle de validation d'une intervention 33
Figure 5.2 — Diagramme de classes du domaine métier 34
Figure 5.4 — Séquence de déclaration d'un frais 35
Figure 5.5 — Activités principales selon le rôle utilisateur 36
Listing 6.1: Validation de la période d'une intervention 41
Listing 6.2: Principe de contrôle avant la création d'un frais 41
Listing 6.3: Dépendance de contrôle des rôles 42
Figure 6.1 — Écran de connexion à l'application 42
Figure 6.2 — Tableau de bord administrateur 43
Figure 6.3 — Création et gestion des employés 44
Figure 6.4 — Écran de planification des interventions 44
Figure 6.5 — Suivi des frais par l'administrateur 45
Figure 6.6 — Tableau de bord de l'employé 46
Figure 6.7 — Consultation des interventions de l'employé 46
Figure 6.8 — Déclaration et suivi des frais par l'employé 47

Liste des tableaux

Tableau 1.1 — Périmètre du projet 5
Tableau 1.2 — Découpage indicatif du stage 6
Tableau 2.1 — Diagnostic du processus initial 9
Tableau 2.2 — Parties prenantes et attentes 9
Tableau 2.3 — Exigences fonctionnelles principales 10
Tableau 2.4 — Contraintes et critères mesurables 11
Tableau 2.5 — Comparaison des approches envisagées 11
Tableau 3.1 — Itérations réalisées pendant le stage 14
Tableau 3.2 — Backlog du projet 15
Tableau 3.3 — Priorisation de type MoSCoW 16
Tableau 3.4 — Principaux risques suivis 17
Tableau 4.1 — Acteurs du système 20
Tableau 4.2 — Matrice des autorisations 21
Tableau 4.3 — Besoins non fonctionnels 22
Tableau 4.4 — Principaux cas limites 27
Tableau 5.1 — Entités persistées 32
Tableau 5.2 — Transformation des relations en base 32
Tableau 5.3 — Menaces principales et mesures prévues 37
Tableau 6.1 — Technologies utilisées dans la version finale 40
Tableau 6.2 — Exemples d'échanges entre l'interface et l'API 47
Tableau 6.3 — Synthèse des ressources de l'API 48
Tableau 7.1 — Résultats des vérifications exécutées 51
Tableau 7.2 — Scénarios de recette fonctionnelle 52
Tableau 7.3 — Matrice de traçabilité 54
Tableau D.1 - Fonctionnalite supplementaire ajoutee 63
Tableau D.2 - Scenarios de verification du module rapports 63
Figure D.1 - Diagramme de cas d'utilisation avec module Rapports 64
Figure D.2 - Diagramme de classes avec ReportService et ReportSummary 65
Figure D.3 - Sequence de consultation des rapports et export CSV 66
Figure D.4 - Activites principales avec rapports statistiques 67

Liste des acronymes

Acronyme	Signification
API	Interface de programmation applicative
CRUD	Création, lecture, modification et suppression
GSI	Gestion des Systèmes d'Information
HTTP	Protocole de transfert hypertexte
JWT	Jeton web JSON utilisé pour l'authentification
ORM	Mapping objet-relationnel
PFE	Projet de fin d'études
REST	Style d'architecture pour les services web
SPA	Application web monopage
UIC	Université Internationale de Casablanca
UML	Langage de modélisation unifié

Introduction générale

Dans le secteur des travaux et des équipements électriques, la bonne coordination des équipes de terrain est indispensable au respect des délais et à la qualité des interventions. Les responsables doivent quotidiennement affecter les employés aux différents chantiers, suivre les périodes d'intervention et contrôler les frais engagés lors des déplacements. Lorsque ces informations sont réparties entre plusieurs fichiers, messages et documents, leur consultation devient difficile et les risques d'erreur augmentent.

C'est dans ce contexte que s'inscrit mon projet de fin d'études, réalisé au sein de l'entreprise CAIDELEC du 4 mai au 4 juillet 2026. L'entreprise souhaitait disposer d'un outil centralisé permettant d'améliorer la planification des interventions et le suivi des dépenses professionnelles des employés. Le projet consiste ainsi à concevoir et à développer une application web adaptée aux besoins des responsables administratifs et des équipes de terrain.

La problématique principale de ce projet peut être formulée de la manière suivante : comment centraliser la planification des interventions et le contrôle des frais de déplacement dans une application simple, tout en garantissant la cohérence des données, la séparation des rôles et la traçabilité des opérations ?

Pour répondre à cette problématique, l'application distingue deux profils. L'administrateur peut gérer les employés et leurs comptes, planifier les interventions, consulter le calendrier, suivre les frais déclarés et exporter certaines données vers Excel. L'employé peut consulter les interventions qui lui sont affectées, déclarer ses frais de transport, de repas ou autres dépenses et joindre une photo justificative.

La réalisation du projet a commencé par l'étude du besoin et l'identification des acteurs, des fonctions attendues et des principales règles de gestion. Ces éléments ont ensuite été représentés à travers des diagrammes UML, notamment les diagrammes de cas d'utilisation, de classes, de séquence et d'activité. Le développement repose sur une architecture séparant l'interface utilisateur, réalisée avec React et Vite, de l'API REST développée avec FastAPI. SQLAlchemy assure l'accès aux données, tandis que SQLite est utilisé pour le stockage de la version livrée. L'authentification est protégée par des jetons JWT et par des contrôles basés sur le rôle de l'utilisateur.

Le présent rapport est organisé en sept chapitres. Le premier présente l'organisme d'accueil et le cadre général du projet. Le deuxième expose l'étude préalable et le cahier des charges. Le troisième décrit la conduite du projet. Le quatrième détaille l'analyse et la spécification des besoins. Le cinquième présente la conception UML et l'architecture technique. Le sixième décrit la réalisation de l'application. Enfin, le septième présente les vérifications effectuées, les résultats obtenus ainsi que les limites de la version actuelle.

Introduction

La gestion des équipes, des interventions et des frais de déplacement constitue un enjeu important pour les entreprises réalisant des travaux sur plusieurs chantiers. La dispersion des informations rend souvent le suivi des activités difficile.

Dans ce contexte, mon projet de fin d'études, réalisé au sein de CAIDELEC du 4 mai au 4 juillet 2026, consiste à développer une application web permettant de centraliser ces différentes opérations.

L'application offre à l'administrateur la possibilité de gérer les employés, planifier les interventions et vérifier les frais. L'employé peut consulter ses missions et déclarer ses dépenses avec un justificatif.

Ce rapport présente l'analyse des besoins, la conception UML, l'architecture technique, la réalisation de l'application ainsi que les tests et les résultats obtenus.

1. Contexte du projet

Les entreprises qui interviennent quotidiennement sur plusieurs chantiers doivent coordonner des équipes mobiles, répartir les missions, suivre l'avancement des travaux et contrôler les dépenses engagées sur le terrain. Ces opérations sont liées : une dépense n'a de sens que si elle peut être rattachée à une intervention, à un employé et à une période déterminée. Lorsque les informations restent dispersées entre des échanges informels et des fichiers bureautiques, la vision d'ensemble devient difficile et la vérification demande davantage de temps.

CAIDELEC, entreprise spécialisée dans les travaux et les équipements électriques, a exprimé le besoin de mieux structurer ce suivi. Pendant le stage réalisé du 4 mai au 4 juillet 2026, j'ai étudié le circuit allant de la préparation d'une intervention à la déclaration des frais correspondants. L'objectif n'était pas de reproduire un simple tableau dans une page web, mais de proposer un enchaînement cohérent d'actions, adapté aux responsabilités de chaque utilisateur.

2. Problématique

La question directrice du projet est la suivante : comment centraliser la planification des équipes de terrain et le contrôle des frais de déplacement dans une application simple à utiliser, tout en garantissant la séparation des rôles, la cohérence des données et la traçabilité des validations ? Cette problématique implique à la fois une dimension organisationnelle, liée au partage de l'information, et une dimension technique, liée à la sécurité des accès et à l'intégrité des relations entre comptes, employés, interventions et dépenses.

3. Objectifs et démarche

Le travail a été organisé autour de quatre objectifs concrets : disposer d'un référentiel des employés et de leurs comptes ; planifier les interventions dans un calendrier partagé ; permettre à chaque employé de consulter uniquement ses propres missions et de déclarer ses frais ; donner à l'administrateur une vue de contrôle, un suivi par statuts et un export Excel. Les choix fonctionnels ont d'abord été traduits en scénarios et en règles de gestion, puis en modèles UML. Le développement a ensuite suivi une architecture séparant l'interface React, l'API FastAPI et la persistance SQLAlchemy/SQLite.

La validation repose sur des scénarios reproductibles : connexion selon le rôle, création d'un employé, planification d'une intervention, consultation côté employé, déclaration d'une dépense, contrôle du plafond et vérification par l'administrateur. Cette démarche permet de relier chaque exigence à une fonction visible de l'application et à un résultat observable pendant la démonstration.

4. Organisation du rapport

Le premier chapitre présente l'organisme d'accueil, le contexte du stage et le périmètre retenu. Le deuxième analyse l'existant et formalise le cahier des charges. Le troisième décrit l'organisation du travail. Les quatrième et cinquième chapitres traitent respectivement de l'analyse fonctionnelle, de la modélisation UML et de l'architecture. Le sixième expose la réalisation à partir d'éléments directement observables dans l'application. Le septième rassemble les vérifications, les résultats et les limites de la version livrée. La conclusion revient enfin sur les apports du projet et sur les évolutions prioritaires.

Chapitre 1

Ce chapitre présente CAIDELEC, l'entreprise d'accueil, ainsi que le contexte dans lequel le projet a été réalisé. Il met en évidence les difficultés liées à la dispersion des informations, au suivi du planning et au contrôle des frais. Il définit enfin les objectifs, le périmètre fonctionnel et les principaux livrables du projet.

Cadre général du projet

1.1 Présentation de l'organisme d'accueil

CAIDELEC est l'organisme d'accueil dans lequel le projet a été réalisé. Son activité implique la mobilisation d'équipes techniques sur différents chantiers ainsi qu'une coordination quotidienne entre les responsables administratifs et les intervenants de terrain. Dans ce contexte, la qualité du planning, la disponibilité des informations et la justification des frais ont un impact direct sur l'organisation des opérations.

Le projet ne cherche pas à remplacer l'ensemble du système d'information de l'entreprise. Il couvre un périmètre ciblé : les employés concernés par les interventions, les affectations par chantier et période, les dépenses déclarées, les justificatifs et la validation par un responsable.

1.2 Contexte et origine du besoin

Avant la mise en place de la solution, le suivi reposait sur un processus peu centralisé. Les responsables devaient rassembler plusieurs informations avant de produire une vision journalière : disponibilité des employés, chantier à desservir, nature des travaux, période de l'intervention et dépenses associées. Cette organisation présentait notamment les limites suivantes :

- une consultation lente du planning quotidien et mensuel ;
- un risque d'erreur ou d'oubli dans les affectations ;
- une traçabilité insuffisante de l'état réel des interventions ;
- une difficulté à contrôler les frais sans justificatif associé ;
- une production manuelle et répétitive des fiches d'affectation ;
- une séparation peu claire entre les actions du responsable et celles de l'employé.

Question directrice

Comment centraliser la planification des interventions et les frais de déplacement dans une application facile à utiliser, tout en garantissant la séparation des rôles et la traçabilité des validations ?

1.3 Objectifs du projet

L'objectif principal consiste à digitaliser le cycle allant de la création d'un employé à la validation d'une intervention et de ses frais. Cet objectif général se décline en plusieurs

objectifs opérationnels :

1. fournir une authentification et une navigation adaptées au rôle ;
2. centraliser les données des employés et leurs comptes utilisateurs ;
3. planifier une intervention sur une période et l'affecter à un employé ;
4. proposer une consultation calendaire et des indicateurs synthétiques ;
5. permettre à l'employé de déclarer l'exécution de sa mission ;
6. saisir les frais de transport, repas et autres dépenses ;
7. associer un justificatif photographique à une dépense ;
8. permettre à l'administrateur de vérifier les interventions et les frais ; 9. exporter les affectations d'une journée dans un fichier Excel structuré.

1.4 Périmètre fonctionnel

Tableau 1.1 — Périmètre du projet

Domaine	Inclus dans la version réalisée	Hors périmètre actuel
Utilisateurs	Authentification, profil administrateur et profil employé	Annuaire complet, multi-entreprise
Employés	Création, consultation, modification, suppression, compte lié, plafond de frais	Paie, compétences et contrats
Interventions	Affectation, période, chantier, nature, observation, statuts	Optimisation automatique des tournées
Frais	Transport, repas, autres, justificatif, contrôle du plafond, validation	Intégration comptable automatique
Pilotage	Tableaux de bord, calendrier et export Excel journalier	Analyse décisionnelle avancée

1.5 Organisation et méthode de travail

Le stage s'est déroulé du 4 mai au 4 juillet 2026. Une démarche itérative a été adoptée : compréhension du besoin, préparation des modèles, réalisation d'un premier noyau fonctionnel, ajout progressif des règles de contrôle et validation à travers les écrans de démonstration.

Tableau 1.2 — Découpage indicatif du stage

Période	Activités principales	Livrables
4–15 mai	Analyse de l'existant, clarification des rôles et identification des données	Besoins, périmètre, backlog initial
18–29 mai	Modélisation UML et conception de l'architecture	Cas d'utilisation, classes, séquences, activités
1–12 juin	Développement de l'API, des modèles et de l'authentification	API FastAPI et base SQLite
15–26 juin	Développement des écrans React et intégration avec l'API	Interfaces administrateur et employé
29 juin–4 juillet	Validation, corrections, préparation de l'export et documentation	Application démontrable, présentation et rapport

1.6 Livrables

Les principaux livrables sont l'application web complète, les sources séparées en frontend et backend, la base de données de démonstration, les modèles UML, la présentation de soutenance et le présent rapport. Le code final constitue la référence pour décrire les fonctionnalités effectivement disponibles.

1.7 Conclusion du chapitre

Ce chapitre a situé le projet dans son contexte professionnel et défini un périmètre centré sur la planification, le suivi et le contrôle. Le chapitre suivant transforme ces objectifs enbesoins précis et en règles de gestion.

Chapitre 2

Ce chapitre analyse le fonctionnement existant afin de déterminer précisément les besoins de l'entreprise. Il identifie deux acteurs principaux : l'administrateur, responsable de l'organisation et du contrôle, et l'employé, chargé de consulter ses missions et de déclarer ses frais. Il présente également les fonctionnalités attendues ainsi que les contraintes techniques, ergonomiques et sécuritaires.

Étude préalable et cahier des charges

2.1 Démarche suivie pour comprendre le besoin

Au début du stage, la difficulté principale n'était pas de choisir un framework. Il fallait d'abord comprendre quelles informations devaient circuler entre le responsable et les employés. Une intervention possède un lieu, une période, une nature de travaux et un intervenant. Les frais ne constituent donc pas un document isolé : ils doivent rester rattachés à cette intervention afin que le responsable sache à quel chantier correspond chaque montant.

L'étude s'est appuyée sur le document de projet transmis, les échanges avec l'encadrement et les exemples de données utilisés pendant la réalisation. J'ai reformulé le besoin en scénarios simples, puis j'ai vérifié pour chacun les données nécessaires, le responsable de l'action et le résultat attendu. Cette étape a fait apparaître deux profils bien distincts : l'administrateur, qui organise et contrôle, et l'employé, qui consulte et déclare.

2.2 Fonctionnement avant la solution

Le suivi d'une équipe mobile peut rapidement se disperser entre plusieurs supports : appels téléphoniques, messages, tableaux bureautiques et justificatifs conservés séparément. Même lorsque chaque support est utile, l'absence d'un point central oblige le responsable à reconstruire l'information avant de prendre une décision. Pour le périmètre étudié, le processus de référence peut être résumé par les étapes suivantes :

1. identifier les employés disponibles et les chantiers à couvrir ;
2. préparer l'affectation en précisant la période et la nature des travaux ;
3. transmettre l'information à l'employé concerné ;
4. attendre le retour sur la réalisation de la mission ;
5. récupérer les montants de transport, de repas et les autres dépenses ;
6. rapprocher les montants du chantier et contrôler le justificatif ; 7. préparer, si nécessaire, une fiche récapitulative pour la journée.

Ce fonctionnement reste possible avec un petit nombre d'interventions, mais il devient fragile lorsque les affectations se multiplient ou changent. Une modification communiquée oralement peut ne pas être reportée dans le tableau. Un justificatif peut être reçu sans référence claire au chantier. Enfin, l'employé n'a pas toujours une vue immédiate sur l'état de validation de sa déclaration.

2.3 Diagnostic de l'existant

Tableau 2.1 — Diagnostic du processus initial

Point observé	Conséquence possible	Réponse attendue
Informations réparties	Temps de recherche et risque de versions différentes	Une base unique accessible par les deux profils
Affectations transmises séparément	Oubli d'une modification ou mauvaise lecture de la période	Une liste personnelle et un calendrier central
Frais sans lien direct avec le planning	Contrôle plus long et difficulté à retrouver le chantier	Rattachement obligatoire du frais à l'intervention
Justificatifs reçus à part	Pièce difficile à rapprocher de la déclaration	Ajout de la photographie dans le même formulaire
Validation peu visible	Relances et incertitude pour l'employé	Statuts « à vérifier » et « vérifié »
Préparation manuelle des fiches	Travail répétitif et présentation variable	Export Excel journalier mis en forme

Le diagnostic ne conduit pas à numériser toutes les activités de CAIDELEC. J'ai retenu le circuit qui apporte un résultat immédiatement visible : la préparation d'une intervention et le suivi de la dépense correspondante. Les fonctions de paie, de comptabilité générale, de gestion des contrats ou d'optimisation des tournées restent volontairement hors du projet.

2.4 Parties prenantes

Tableau 2.2 — Parties prenantes et attentes

Partie prenante	Relation au projet	Attentes principales
Responsable CAIDELEC	Commanditaire métier	Disposer d'une vue globale, affecter rapidement les équipes et contrôler les déclarations.
Employé de terrain	Utilisateur quotidien	Retrouver ses missions sans ambiguïté et déclarer ses dépenses simplement.
Encadrement pro-fessionnel	Validation du besoin	Vérifier l'adéquation entre la solution et l'organisation de l'entreprise.
Encadrement académique	Suivi méthodologique	Évaluer l'analyse, la conception, la qualité de réalisation et la prise de recul.
Stagiaire développeur	Conception et réalisation	Livrer une application cohérente, compréhensible et démontrable dans la période prévue.

2.5 Cahier des charges fonctionnel

Le cahier des charges a été organisé par domaines afin de conserver un lien direct entre l'écran, l'API et les données. Chaque exigence reçoit un identifiant qui sera repris dans la matrice de traçabilité du chapitre de validation.

Tableau 2.3 — Exigences fonctionnelles principales

ID	Domaine	Exigence	Priorité
F01	Authentification	Permettre la connexion par nom d'utilisateur et mot de passe.	Haute
F02	Autorisation	Adapter les pages et les données au rôle administrateur ou employé.	Haute
F03	Employés	Créer, consulter, modifier et supprimer une fiche employé.	Haute
F04	Comptes	Lier un compte utilisateur à la fiche de l'employé.	Haute
F05	Interventions	Affecter un employé à un chantier sur une période valide.	Haute
F06	Interventions	Consulter et mettre à jour les affectations depuis l'espace administrateur.	Haute
F07	Espace employé	Limiter la liste aux interventions de l'utilisateur connecté.	Haute
F08	Suivi	Permettre à l'employé de déclarer la réalisation d'une intervention.	Haute
F09	Frais	Saisir les montants de transport, repas et autres frais.	Haute
F10	Frais	Joindre une photographie justificative à la déclaration.	Moyenne
F11	Contrôle	Empêcher le dépassement du plafond attribué au compte.	Haute
F12	Validation	Permettre au responsable de vérifier intervention et dépense.	Haute
F13	Pilotage	Afficher des indicateurs et un calendrier mensuel.	Moyenne
F14	Export	Produire une fiche Excel des affectations d'une journée.	Moyenne

ID	Domaine	Exigence	Priorité
F15	Traçabilité	Afficher le statut actuel des interventions et des frais.	Haute

2.6 Contraintes non fonctionnelles

Le projet devait rester utilisable depuis un navigateur sans installation sur le poste de l'employé. Le délai de deux mois imposait également de privilégier une architecture connue, légère et facile à démontrer. La base SQLite répond à ce besoin pour la version de stage, tout en laissant la couche SQLAlchemy préparer une migration ultérieure vers une base serveur.

Tableau 2.4 — Contraintes et critères mesurables

Thème	Contrainte retenue	Critère de contrôle
Simplicité	Limiter le nombre d'actions nécessaires pour les opérations courantes	Formulaires regroupés et navigation stable
Sécurité	Ne jamais faire confiance au seul filtrage de l'interface	Vérification du JWT, du rôle et de la propriété côté API
Qualité des données	Refuser les dates incohérentes et les références absentes	Validation Pydantic et contrôles avant insertion
Lisibilité	Conserver une identité visuelle commune aux deux espaces	Même structure, couleurs et composants de tableau
Maintenabilité	Éviter un fichier unique regroupant toute la logique	Routeurs, modèles, schémas et pages séparés
Portabilité	Préparer une exécution indépendante de la machine de développement	Dockerfiles frontend et backend

2.7 Étude des solutions possibles

Trois approches ont été envisagées. La première consistait à améliorer uniquement un fichier Excel partagé. Elle aurait réduit le temps de développement, mais n'aurait pas apporté de contrôle d'accès fiable ni de séparation entre les données globales et personnelles. La deuxième était une application monolithique rendue côté serveur. Cette option était adaptée à un outil interne, mais moins proche de l'expérience interactive souhaitée pour le calendrier et les tableaux de bord. La troisième, finalement retenue, sépare une interface React et une API FastAPI.

Tableau 2.5 — Comparaison des approches envisagées

Critère	Excel	Web monolithe	SPA + API	Lecture
Contrôle des rôles	Faible	Bon	Bon	Exigence indispensable
Utilisation terrain	Moyenne	Bonne	Bonne	Accès par navigateur
Évolution vers mobile	Faible	Moyenne	Bonne	API réutilisable
Rapidité initiale	Très bonne	Bonne	Moyenne	Coût de séparation des couches
Maintenabilité	Faible	Bonne	Bonne	Responsabilités identifiables
Export bureautique	Natif	À développer	À développer	Couvert avec ExcelJS

Le choix SPA/API n'a pas été fait pour multiplier les technologies. Il répond surtout au besoin de disposer d'une même API pour deux espaces dont les données et les droits diffèrent. L'API devient le point de contrôle, tandis que React se charge de présenter un parcours plus direct à chaque utilisateur.

2.8 Critères de réception

La version est considérée démontrable si un administrateur peut créer un employé, planifier une intervention et retrouver cette affectation dans le calendrier ; si l'employé connecté ne voit que ses propres données ; et si une dépense peut être déclarée puis contrôlée par l'administrateur. Le refus d'une date incohérente, d'une ressource appartenant à un autre employé ou d'un frais dépassant le plafond fait partie du résultat attendu, au même titre qu'un scénario nominal réussi.

2.9 Conclusion du chapitre

Cette étude préalable a transformé une idée générale de gestion du planning en un périmètre vérifiable. Elle a aussi permis de justifier le lien central entre employé, intervention et dépense. Le chapitre suivant présente la manière dont ces éléments ont été organisés pendant la période de stage.

Chapitre 3

Ce chapitre décrit l'organisation du travail pendant la période du stage. Le développement a été réalisé progressivement, en commençant par l'authentification, puis la gestion des employés, des interventions et des frais. Cette méthode a permis de prioriser les fonctionnalités importantes, de suivre les risques et de vérifier régulièrement l'avancement du projet.

Conduite du projet

3.1 Organisation générale

Le stage s'est déroulé du 4 mai au 4 juillet 2026. Avec une période aussi courte, un déroulement entièrement séquentiel aurait été risqué : attendre la fin de toute la documentation avant de commencer le développement aurait laissé peu de temps pour confronter les modèles à l'application. J'ai donc travaillé par petites itérations. Chaque itération partait d'un scénario métier, ajoutait les données et les routes nécessaires, puis se terminait par un écran utilisable.

Le premier fil fonctionnel choisi a été la connexion, car toutes les autres pages en dépendent. Le second a relié la création d'un employé à son compte utilisateur. Le troisième a couvert l'intervention de sa création à sa consultation par l'employé. Enfin, le quatrième a ajouté la déclaration et la vérification des frais. Le calendrier, les indicateurs et l'export Excel sont venus compléter ce noyau.

3.2 Découpage en itérations

Tableau 3.1 — Itérations réalisées pendant le stage

Période	Objectif	Travaux menés	Résultat
4–15 mai	Comprendre le circuit métier	Étude du besoin, recensement des données, clarification des rôles et du périmètre	Cahier des charges initial
18–29 mai	Stabiliser la conception	Diagrammes de cas d'utilisation, classes, séquence et activité sous StarUML	Modèle partagé du système
1–7 juin	Construire le socle backend	Modèles SQLAlchemy, schémas Pydantic, base SQLite et structure des routeurs	API structurée
8–14 juin	Sécuriser les accès	Hachage des mots de passe, JWT, dépendances de rôle et contrôles de propriété	Connexion et routes protégées
15–21 juin	Réaliser l'espace administrateur	Employés, interventions, tableau de bord, calendrier et suivi des frais	Parcours de planification

Période	Objectif	Travaux menés	Résultat
22–28 juin	Réaliser l'espace employé	Missions personnelles, déclaration, justificatif et statut 10	Parcours terrain

Période	Objectif	Travaux menés	Résultat
29 juin–4 juillet	Consolider la livraison	Export Excel, corrections, vérifications techniques, captures et documentation	Version de démonstration

3.3 Backlog fonctionnel

Les besoins ont été rédigés sous forme de courtes histoires utilisateur. Cette formulation m'a aidé à vérifier qu'une fonction correspondait toujours à une personne et à un résultat, et pas seulement à un écran à développer.

Tableau 3.2 — Backlog du projet

ID	Profil	Histoire utilisateur	Lot
US01	Utilisateur	En tant qu'utilisateur, je veux me connecter afin d'accéder à mon espace.	Socle
US02	Administrateur	Je veux créer un employé et son compte pour l'intégrer au planning.	Équipe
US03	Administrateur	Je veux modifier les coordonnées et le plafond afin de garder les données à jour.	Équipe
US04	Administrateur	Je veux affecter un employé à un chantier sur une période donnée.	Planning
US05	Administrateur	Je veux parcourir un calendrier mensuel pour repérer les jours planifiés.	Planning
US06	Administrateur	Je veux exporter les affectations du jour pour les diffuser sous une forme homogène.	Export
US07	Employé	Je veux voir uniquement mes interventions afin de connaître mes missions.	Terrain
US08	Employé	Je veux indiquer qu'une mission est terminée pour déclencher son contrôle.	Terrain
US09	Employé	Je veux saisir mes frais par catégorie et les rattacher à une intervention.	Frais

ID	Profil	Histoire utilisateur	Lot
US10	Employé	Je veux ajouter une photo du justificatif pour compléter ma déclaration.	Frais
US11	Administrateur	Je veux contrôler une intervention avant de la considérer comme vérifiée.	Contrôle
US12	Administrateur	Je veux consulter le justificatif et vérifier le frais correspondant.	Contrôle
US13	Chaque profil	Je veux voir un tableau de bord adapté à mes responsabilités.	Pilotage

3.4 Priorisation

La priorité la plus élevée a été donnée aux fonctions qui ferment le cycle métier : authentification, gestion des employés, planification, consultation personnelle et frais. Sans l'une de ces briques, la démonstration aurait présenté plusieurs écrans sans parcours complet. Les indicateurs et l'export ont été traités ensuite. Des idées comme les notifications, l'historique d'audit ou la gestion des absences ont été conservées comme perspectives, car elles demandaient des données et des règles supplémentaires.

Tableau 3.3 — Priorisation de type MoSCoW

Niveau	Éléments
Must have	Connexion, rôles, employés, interventions, consultation personnelle, déclaration et validation des frais.
Should have	Calendrier, tableau de bord, justificatif photographique, contrôle du plafond et export Excel.
Could have	Notifications, statistiques avancées, recherche multicritère et génération d'un rapport mensuel.
Won't have pour cette version	Paie, comptabilité, géolocalisation, optimisation automatique des tournées et application mobile native.

3.5 Gestion des risques

La conduite du projet a également consisté à identifier tôt les points qui pouvaient bloquer la démonstration. Par exemple, la gestion des droits devait être testée dès les premières routes ; la laisser pour la fin aurait obligé à reprendre l'ensemble des accès. De même, l'export Excel dépend d'une bibliothèque relativement lourde, raison pour laquelle il a été isolé des parcours indispensables.

Tableau 3.4 — Principaux risques suivis

Risque	Probabilité	Impact	Réponse adoptée
Périmètre trop large	Moyenne	Fort	Concentration sur deux profils et quatre entités principales.
Confusion entre rôle et fonction profes-sionnelle	Moyenne	Fort	Champ de rôle technique séparé de la fonction de l'employé.
Accès aux données d'un autre employé	Moyenne	Fort	Contrôle de propriété effectué côté API.
Dates d'intervention incohérentes	Moyenne	Moyen	Validation des schémas avant l'enregistrement.
Dépassement du plafond	Moyenne	Moyen	Calcul du cumul au moment de la déclaration.
Justificatif volumineux	Moyenne	Moyen	Solution simple pour la démonstration et recommandation d'un stockage dédié.
Dépendances frontend non repro-ductibles	Faible	Moyen	Vérification de l'installation et resynchronisation du verrouillage npm.
Retard de documen-tation	Moyenne	Moyen	Captures et modèles conservés au fur et à mesure de la réalisation.

3.6 Suivi de la qualité

À chaque ajout, j'ai vérifié quatre points : le comportement avec le bon profil, le refus avec le mauvais profil, la cohérence des données enregistrées et la lisibilité du message retourné en cas d'erreur. Côté interface, la navigation latérale et les composants de formulaire ont été réutilisés pour limiter les différences inutiles entre les pages. Côté backend, la séparation en routeurs a permis de relire les règles domaine par domaine.

Les diagrammes StarUML n'ont pas seulement servi à illustrer le rapport. Le diagramme de classes a fixé les liens entre les quatre entités, tandis que le diagramme de séquence a permis de placer les contrôles de propriété et de plafond avant l'écriture en base. Lorsqu'un écart apparaissait entre un diagramme et le code, la version réellement livrée était analysée afin de mettre la documentation à jour.

3.7 Bilan de l'organisation

Cette organisation itérative a permis d'obtenir assez tôt un parcours fonctionnel, puis de l'enrichir sans perdre la cohérence générale. Elle a aussi montré une limite : certaines améliorations techniques, comme les migrations versionnées ou une suite de tests automatisés complète, n'ont pas pu être menées aussi loin que

les fonctionnalités visibles. Ce choix est assumé dans le cadre du stage, mais ces éléments restent nécessaires avant une mise en production durable.

3.8 Conclusion du chapitre

Le découpage par parcours a servi de lien entre le cahier des charges, les modèles UML et le code. Le chapitre suivant entre dans la spécification détaillée des acteurs, des règles de gestion et des cas d'utilisation.

Chapitre 4

Ce chapitre présente les besoins fonctionnels et non fonctionnels de l'application ainsi que les droits accordés à chaque utilisateur. Il détaille les fonctions liées à l'authentification, aux employés, aux interventions, aux frais et à l'export des données. Les règles de gestion et les cas d'utilisation permettent de représenter clairement les interactions entre les acteurs et le système.

Analyse et spécification des besoins

4.1 Identification des acteurs

Le système distingue deux acteurs humains. Cette séparation structure l'interface, les autorisations de l'API et l'accès aux données.

Tableau 4.1 — Acteurs du système

Acteur	Responsabilités
Administrateur	Gérer les employés et leurs comptes, planifier et modifier les interventions, consulter l'ensemble des données, vérifier les interventions et les frais, consulter le calendrier et générer les affectations Excel.
Employé	Consulter uniquement ses interventions, déclarer une mission réalisée, saisir ou modifier ses frais, joindre un justificatif et suivre le statut de ses déclarations.

4.2 Besoins fonctionnels

4.2.1 Authentification et contrôle d'accès

L'utilisateur se connecte à l'aide d'un nom d'utilisateur et d'un mot de passe. Après validation, l'API renvoie un jeton JWT contenant l'identifiant logique du compte, son rôle et, le cas échéant, l'identifiant de l'employé lié. Les routes protégées vérifient ce jeton et le rôle attendu.

4.2.2 Gestion des employés

L'administrateur peut créer une fiche employé avec le nom complet, la fonction, le téléphone et un plafond de frais. La création produit également un compte utilisateur lié. Un nom d'utilisateur peut être fourni ou généré à partir du nom de l'employé ; un mot de passe initial est associé au compte. Les données peuvent ensuite être modifiées ou supprimées.

4.2.3 Gestion des interventions

Une intervention comporte un employé, un chantier, une nature de travaux, une observation, une date de début, une date de fin et un statut. L'administrateur crée l'affectation. L'employé ne voit que les interventions liées à son compte et peut les déclarer terminées.

L'administrateur réalise ensuite le contrôle final.

4.2.4 Gestion des frais

Une dépense est rattachée à une intervention et répartie entre transport, repas et autres frais. L'employé ne peut déclarer une dépense que pour l'une de ses propres interventions. Le total cumulé est comparé au

plafond associé à son compte. Après ajout d'un justificatif photographique, l'administrateur peut valider la dépense.

4.2.5 Tableaux de bord et export

Les tableaux de bord affichent des indicateurs et un calendrier mensuel. Le contenu varie selon le rôle : vue globale pour l'administrateur, vue personnelle pour l'employé. Une vue journalière permet d'exporter une fiche Excel mise en forme avec les affectations du jour.

4.3 Matrice des droits

Tableau 4.2 — Matrice des autorisations

Fonction	Administrateur	Employé
Se connecter et consulter son profil	Oui	Oui
Gérer les employés et leurs comptes	Oui	Non
Créer, modifier ou supprimer une intervention	Oui	Non
Consulter toutes les interventions	Oui	Non
Consulter ses propres interventions	–	Oui
Déclarer une intervention réalisée	Non	Oui
Vérifier une intervention	Oui	Non
Consulter tous les frais	Oui	Non
Créer ou modifier ses propres frais	Non	Oui
Vérifier un frais avec justificatif	Oui	Non
Exporter les affectations Excel	Oui	Non

4.4 Règles de gestion

RG1. Un compte de rôle employé doit être lié à une fiche employé pour accéder aux données personnelles.

RG2. La date de fin d'une intervention doit être supérieure ou égale à sa date de début. **RG3.** Seul l'administrateur peut créer et affecter une intervention.

RG4. Un employé ne peut consulter ou modifier que les interventions et les frais qui lui appartiennent.

RG5. Une dépense doit être liée à une intervention existante.

RG6. Le cumul des dépenses de l'employé ne doit pas dépasser son plafond lorsqu'un plafond est défini.

RG7. La modification d'une dépense remet son statut à « à vérifier ».

RG8. Un frais ne peut être vérifié par l'administrateur que si un justificatif est présent.

RG9. Les états d'une intervention suivent le cycle « planifiée », « à vérifier », puis « vérifiée ».

4.5 Besoins non fonctionnels

Tableau 4.3 — Besoins non fonctionnels

Critère	Exigence
Ergonomie	Interface claire, navigation stable et formulaires compréhensibles sur les écrans courants.
Sécurité	Mots de passe hachés, jeton d'accès limité dans le temps, routes protégées et contrôle systématique des rôles.
Intégrité	Validation des dates, contrôle des références et messages HTTP cohérents en cas d'erreur.
Maintenabilité	Séparation frontend/backend, routeurs par domaine, modèles et schémas distincts.
Performance	Temps de réponse adapté à un usage interne et limitation des chargements inutiles.
Portabilité	Exécution avec un navigateur récent ; composants applicatifs conteneurisables.
Traçabilité	Conservation des statuts et des liens entre utilisateur, employé, intervention et dépense.

4.6 Diagramme de cas d'utilisation

Le diagramme de la figure 4.1, réalisé sous StarUML, présente les fonctionnalités par acteur. La connexion conditionne l'accès aux fonctions protégées. Les relations *include* indiquent que la création d'un compte accompagne la gestion d'un employé et que le justificatif complète la déclaration d'un frais.

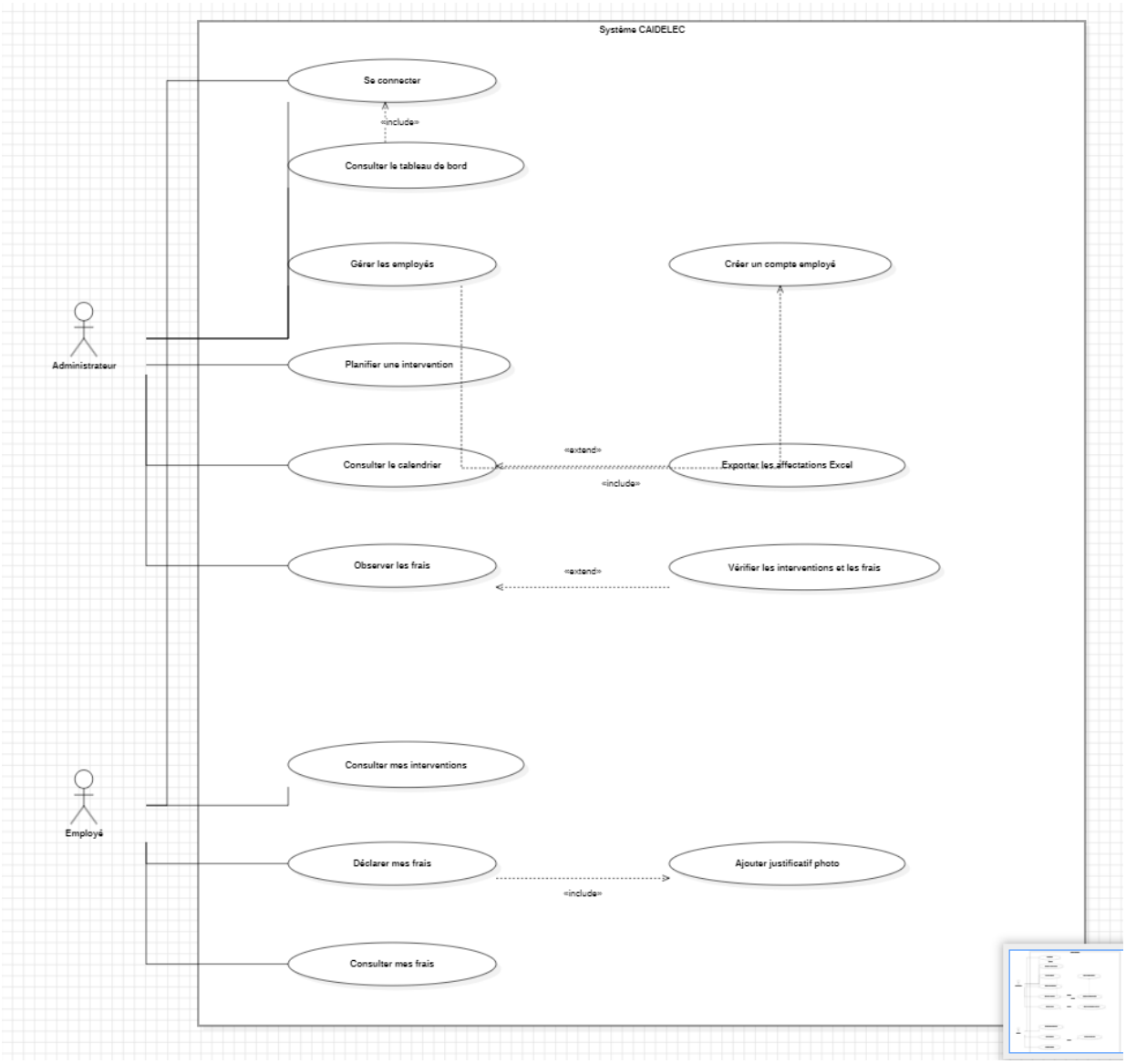


Figure 4.1 — Diagramme des cas d'utilisation du système CAIDELEC

4.7 Spécification de cas d'utilisation essentiels

4.7.1 CU1 – Se connecter

Acteurs	Administrateur, employé
Précondition	Le compte existe dans la base.
Scénario nominal	L'utilisateur saisit ses identifiants ; l'API vérifie le mot de passe ; un jeton JWT est créé ; l'interface redirige vers le tableau de bord associé au rôle.
Exceptions	Identifiant inconnu, mot de passe incorrect ou jeton expiré : l'accès est refusé.

Postcondition Une session applicative authentifiée est ouverte.

4.7.2 CU2 – Planifier une intervention

Acteur	Administrateur
Préconditions	Administrateur authentifié ; employé existant.
Scénario nominal	L'administrateur choisit l'employé, saisit le chantier, la nature, l'observation et la période ; le système valide les dates et enregistre l'intervention avec le statut « planifiée ».
Exceptions	Employé inexistant ou date de fin antérieure à la date de début.
Postcondition	L'intervention apparaît dans le calendrier et dans la liste de l'employé.

4.7.3 CU3 – Déclarer un frais

Acteur	Employé
Préconditions	Employé authentifié ; intervention lui appartenant.
Scénario nominal	L'employé choisit l'intervention, saisit les montants, joint éventuellement une photo et confirme ; le système contrôle le plafond puis enregistre le frais « à vérifier ».
Exceptions	Intervention d'un autre employé, intervention inexistante ou plafond dépassé.
Postcondition	Le frais est visible par l'employé et par l'administrateur.

4.7.4 CU4 – Vérifier un frais

Acteur	Administrateur
Préconditions	Administrateur authentifié ; frais existant avec justificatif.
Scénario nominal	L'administrateur consulte la déclaration et son justificatif, puis confirme la vérification.
Exception	Aucun justificatif n'est associé : la validation est refusée.
Postcondition	Le statut du frais devient « vérifié ».

4.7.5 CU5 – Créer un employé et son compte

Acteur	Administrateur
Préconditions	Administrateur authentifié ; nom d'utilisateur non encore utilisé.
Déclencheur	Le responsable souhaite intégrer une nouvelle personne au planning.
Scénario nominal	L'administrateur saisit le nom complet, la fonction, le téléphone, la limite de frais ainsi que les identifiants du compte ; le système crée la fiche employé puis le compte associé.
Exceptions	Nom d'utilisateur déjà présent, information obligatoire absente ou valeur de plafond incorrecte.
Postcondition	L'employé apparaît dans la liste et peut ouvrir son espace avec le compte créé.

4.7.6 CU6 – Consulter le calendrier et exporter une journée

Acteur	Administrateur
Préconditions	Administrateur authentifié ; interventions enregistrées.
Déclencheur	Le responsable prépare ou contrôle les affectations d'une date donnée.
Scénario nominal	L'administrateur parcourt le mois, sélectionne une journée, consulte les interventions présentes puis demande l'export Excel.
Alternative	Si aucune intervention n'existe pour la date, la vue reste vide et aucun contenu métier n'est ajouté au fichier.
Postcondition	Un classeur daté est téléchargé avec les informations disponibles pour la journée.

4.8 Scénarios détaillés par profil

4.8.1 Journée type de l'administrateur

Après la connexion, l'administrateur consulte d'abord le tableau de bord. Les indicateurs lui donnent une lecture rapide du nombre d'employés, des interventions enregistrées et du total des frais. Il peut ensuite se rendre sur la page « Interventions » pour préparer une nouvelle affectation. Le choix de l'employé n'est pas une saisie libre : il provient des fiches déjà enregistrées, ce qui évite de créer une intervention sans personne correspondante.

Une fois l'affectation sauvegardée, elle devient visible dans le calendrier et dans l'espace personnel de l'employé. Plus tard, lorsque l'employé déclare la mission et ses frais, l'administrateur retrouve les lignes dans les vues de contrôle. La vérification n'efface pas la déclaration : elle modifie son statut, ce qui conserve une trace simple de l'avancement.

4.8.2 Journée type de l'employé

L'employé arrive sur un espace plus réduit. Il ne dispose ni du menu de gestion des personnes ni du formulaire de planification. Le tableau de bord reprend uniquement ses interventions et ses frais. Cette différence n'est pas seulement visuelle : le backend reconstruit le périmètre autorisé à partir du compte lié dans le JWT.

Pour déclarer une dépense, l'employé choisit une de ses interventions, répartit les montants par catégorie puis joint le justificatif. Le système additionne les montants et les compare à la limite définie pour son compte. Une fois la déclaration acceptée, elle apparaît avec un statut en attente de contrôle. L'employé peut ainsi suivre le résultat sans demander au responsable si la ligne a déjà été traitée.

4.9 Cas limites pris en compte

Tableau 4.4 — Principaux cas limites

Situation	Comportement attendu
Compte employé sans fiche liée	Accès aux données personnelles refusé avec une erreur explicite.
Date de fin antérieure à la date de début	Intervention non enregistrée.
Employé supprimé alors que des données sont liées	Opération à contrôler afin de préserver l'intégrité des références.
Intervention demandée par un autre employé	Réponse d'interdiction, même si l'identifiant est connu.
Montant négatif ou non numérique	Rejet par la validation des données.
Cumul supérieur au plafond	Déclaration refusée et limite rappelée à l'utilisateur.
Frais modifié après une première vérification	Retour du statut à « à vérifier ».
Vérification sans justificatif	Action refusée jusqu'à l'ajout de la pièce.
Jeton absent, incorrect ou expiré	Redirection vers la connexion et accès API refusé.

4.10 Glossaire métier

Employé
Personne susceptible d'être affectée à un chantier. Sa fonction professionnelle ne doit pas être confondue avec le rôle technique du compte.

Intervention
Mission planifiée pour un employé, sur un chantier et une période déterminés.

Chantier
Lieu ou site sur lequel les travaux sont réalisés.

Frais
Dépense déclarée à propos d'une intervention, ventilée en transport, repas et autres.

Situation	Comportement attendu
Justificatif	Photographie jointe à une dépense afin de permettre son contrôle.
Plafond	Montant maximal autorisé pour le cumul des frais de l'employé.
Vérification	Action du responsable qui confirme le contrôle d'une intervention ou d'un frais.

4.10 Synthèse des besoins retenus

L'analyse confirme que la planification et les frais doivent être traités dans un même flux. L'intervention constitue l'élément de référence : elle relie l'employé, le chantier, la période, le statut d'exécution et les dépenses déclarées. Cette règle structure le modèle de données et les contrôles appliqués par l'API.

4.11 Conclusion du chapitre

L'analyse met en évidence un cycle métier court mais contrôlé, avec des responsabilités distinctes. Les règles définies servent de base à la conception détaillée présentée dans le chapitre suivant.

Chapitre 5

Ce chapitre présente l'architecture technique de la solution, composée d'une interface React, d'une API FastAPI et d'une base de données SQLite gérée avec SQLAlchemy. Les diagrammes UML illustrent la structure des données, le déroulement des activités et les échanges entre les composants. La sécurité est assurée par l'authentification JWT et la vérification des rôles.

Conception et architecture

5.1 Architecture générale

La solution adopte une architecture découplée. Le navigateur exécute une application

React qui communique avec une API REST FastAPI par des requêtes HTTP via Axios. FastAPI délègue l'accès aux données à SQLAlchemy, qui persiste les entités dans SQLite. Le jeton JWT accompagne les requêtes protégées.

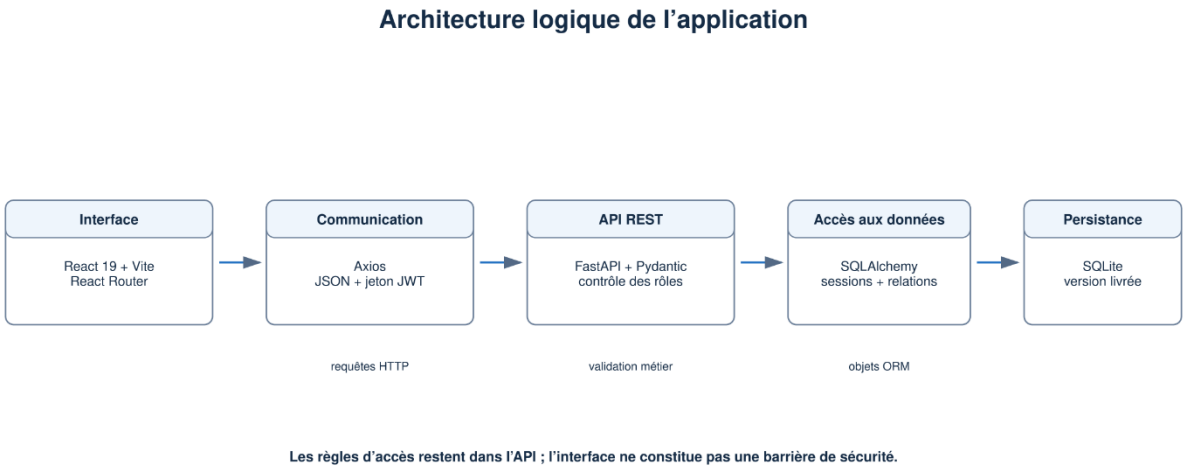


Figure 5.1 — Architecture logique de l'application

Cette séparation présente trois avantages principaux. Premièrement, le frontend peut évoluer sans modifier le stockage. Deuxièmement, l'API centralise les règles d'accès et peut être utilisée par un autre client. Troisièmement, les responsabilités techniques sont identifiables : présentation, services, règles métier et persistance.

5.1.1 Flux d'une requête protégée

Après la connexion, le client ajoute le jeton dans l'en-tête Authorization: Bearer.

FastAPI décode ce jeton avant l'exécution de la route. La dépendance de sécurité vérifie ensuite le rôle et, pour les données personnelles, le lien entre le compte et l'employé. Ce n'est qu'après ces contrôles que la session SQLAlchemy est utilisée pour lire ou modifier la base.

Cette organisation était importante pour le projet. Un simple masquage du menu

« Employés » dans React n'empêcherait pas un utilisateur de construire lui-même une requête HTTP. Le contrôle décisif doit donc rester dans l'API, indépendamment de l'écran à l'origine de la demande.

5.2 Organisation du backend

Le backend est structuré autour de six responsabilités :

- main.py initialise FastAPI, la base, la migration légère SQLite et CORS ;
- database.py configure le moteur SQLAlchemy et les sessions ;
- models.py décrit les tables et relations ;
- schemas.py définit les objets d'entrée/sortie et la validation ;
- auth.py regroupe le hachage, la création JWT et les dépendances de sécurité ;
- le dossier routers sépare les routes d'authentification, d'employés, d'interventions et de frais.

5.3 Organisation du frontend

Le frontend est une SPA React. Le routeur protège les pages et dirige l'utilisateur vers l'espace autorisé. Le composant de mise en page construit la barre latérale et le bandeau utilisateur. Les pages sont organisées par domaine : connexion, tableau de bord, employés, interventions, détail d'une journée et frais. Une instance Axios commune ajoute le jeton Bearer aux requêtes et redirige l'utilisateur en cas de réponse non autorisée.

5.3.1 Découpage des responsabilités côté client

Les pages conservent la gestion des formulaires et de l'affichage, tandis que la communication HTTP est regroupée autour d'une instance Axios. Le routeur choisit le composant à afficher et la mise en page détermine les entrées de navigation selon le rôle. Ce découpage évite de répéter la construction de l'en-tête d'authentification dans chaque écran.

Le stockage local du jeton facilite la démonstration, mais il impose une vigilance face aux scripts injectés dans la page. Pour une évolution de production, un cookie HttpOnly protégé par Secure et SameSite, accompagné d'une stratégie contre les requêtes intersites, constituerait une solution à étudier.

5.4 Modèle de données

Le modèle conceptuel comporte quatre entités principales. User porte les informations de connexion et peut être lié à un Employee. Un employé possède plusieurs interventions.

Chaque intervention peut avoir plusieurs dépenses.

5.4.1 Dictionnaire synthétique des entités

Tableau 5.1 — Entités persistées

Entité	Identifiant / liens	Responsabilité
User	id, employee_id facul-tatif	Authentification, rôle, plafond de frais et lien vers l'employé.
Employee	id	Informations professionnelles et rattachement aux interventions.
Intervention	id, employee_id	Affectation à un chantier sur une période avec un état de suivi.
Expense	id, intervention_id	Montants, justificatif et statut de vérification.

5.4.2 Lecture des associations

Le lien facultatif entre User et Employee permet de représenter le compte administrateur, qui n'a pas besoin d'une fiche employé, et les comptes terrain, qui doivent être rattachés à une personne. Un employé peut recevoir plusieurs interventions au cours du temps. L'intervention constitue ensuite le point de rattachement des frais : cette décision évite une dépense sans chantier ou sans période identifiable.

Le modèle utilise des clés étrangères explicites employee_id et intervention_id. Dans la version SQLite, il faut veiller à activer et respecter les contraintes de référence. Pour une base serveur, les règles de suppression devront être définies avec précision : conserver l'historique, empêcher la suppression ou appliquer une suppression logique sont préférables à une suppression en cascade non contrôlée.

5.4.3 Passage du modèle UML au modèle relationnel

Tableau 5.2 — Transformation des relations en base

Relation UML	Implémentation	Justification
User – Employee	Clé étrangère facultative dans users	Un administrateur peut exister sans fiche employé.

Relation UML	Implémentation	Justification
Employee – Intervention	interventions. employee_id obligatoire	Toute mission doit avoir un destinataire.
Intervention – Expense	expenses. intervention_id obligatoire	Toute dépense doit être expliquée par une mission.
User – plafond	Attribut expense_limit	La limite est vérifiée pour le compte employé authentifié.

5.5 Cycle de vie des objets

Le cycle d'une intervention repose sur trois statuts. L'administrateur crée une intervention « planifiée ». L'employé la déclare terminée, ce qui la place « à vérifier ». L'administrateur effectue le contrôle et la marque « vérifiée ».

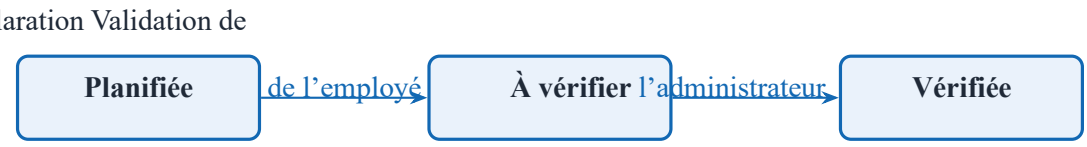


Figure 5.3 — Cycle de validation d'une intervention

La dépense suit une logique proche : elle est créée « à vérifier » et passe à « vérifiée » après contrôle du justificatif. Toute modification d'une dépense réinitialise son statut afin d'éviter qu'une information modifiée reste considérée comme validée.

5.6 Diagramme de classe

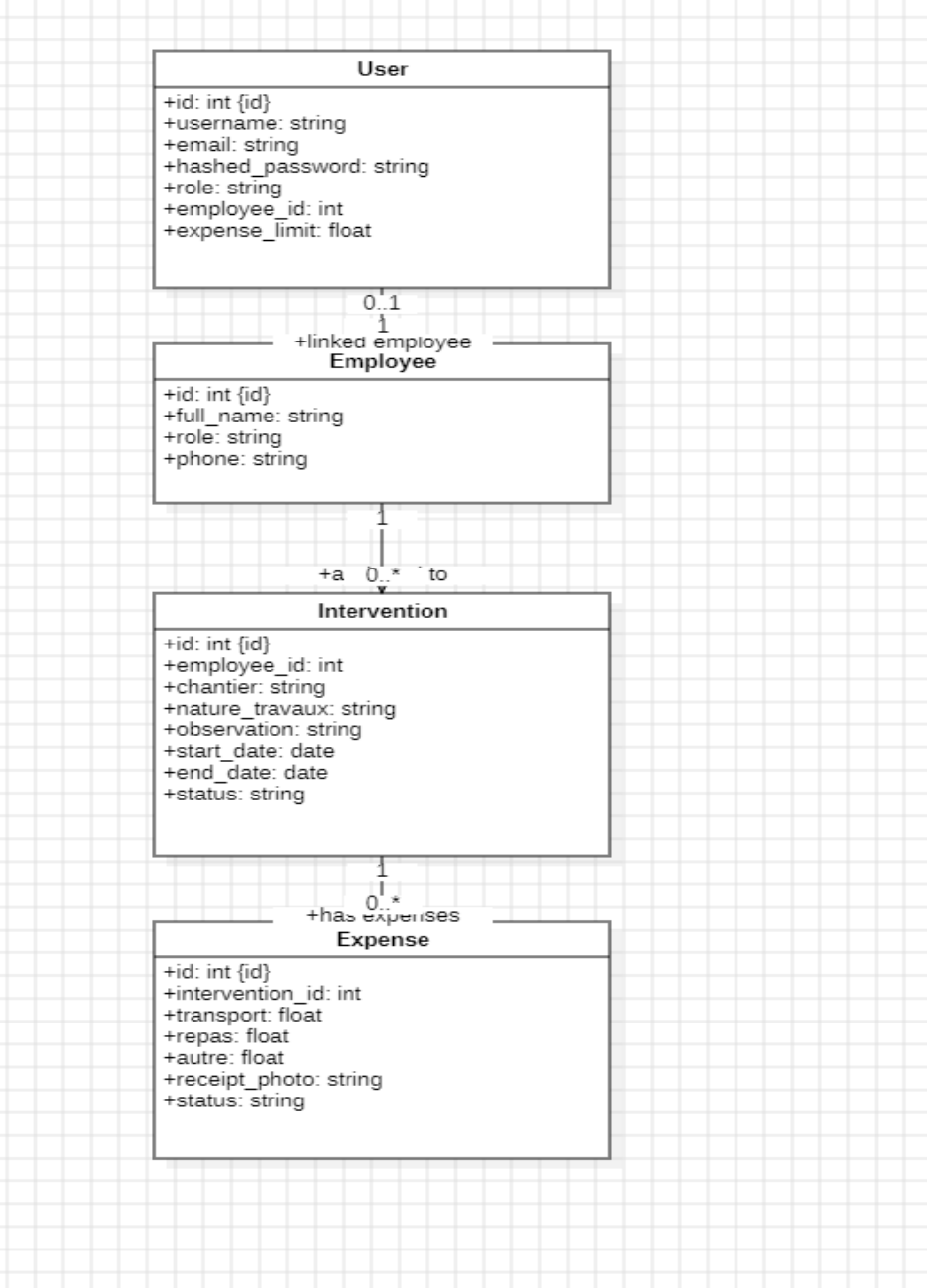


Figure 5.2 — Diagramme de classes du domaine métier

5.7 Diagramme de séquence

La figure 5.4 illustre la déclaration d'un frais et sa vérification. Le backend contrôle l'appartenance de l'intervention, le plafond et la présence du justificatif avant les changements d'état.

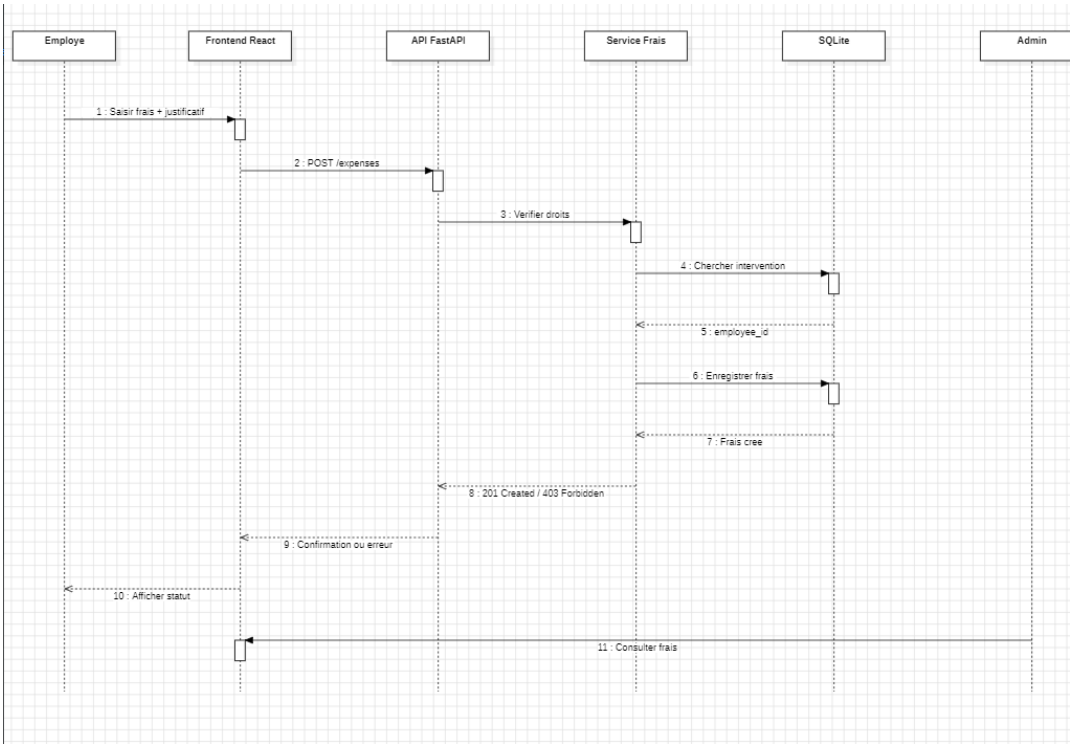


Figure 5.4 — Séquence de déclaration d'un frais

Le diagramme montre que la recherche de l'intervention précède l'enregistrement. Ce point peut sembler secondaire, mais il empêche de créer une dépense en utilisant directement l'identifiant d'une mission appartenant à quelqu'un d'autre. Le retour 201

Created correspond au scénario nominal ; une réponse 403 Forbidden est utilisée lorsque la propriété de la ressource n'est pas respectée.

Lorsque l'administrateur consulte ensuite les frais, l'API renvoie la liste globale autorisée. La vérification constitue une requête distincte afin de conserver une intention claire et de ne pas mélanger une modification du montant avec une validation administrative.

5.7 Diagramme d'activité

Le diagramme d'activité présente le déroulement global de l'application, depuis la connexion jusqu'aux opérations proposées selon le rôle de l'utilisateur. Les couloirs distinguent les actions du système, de l'administrateur et de l'employé.

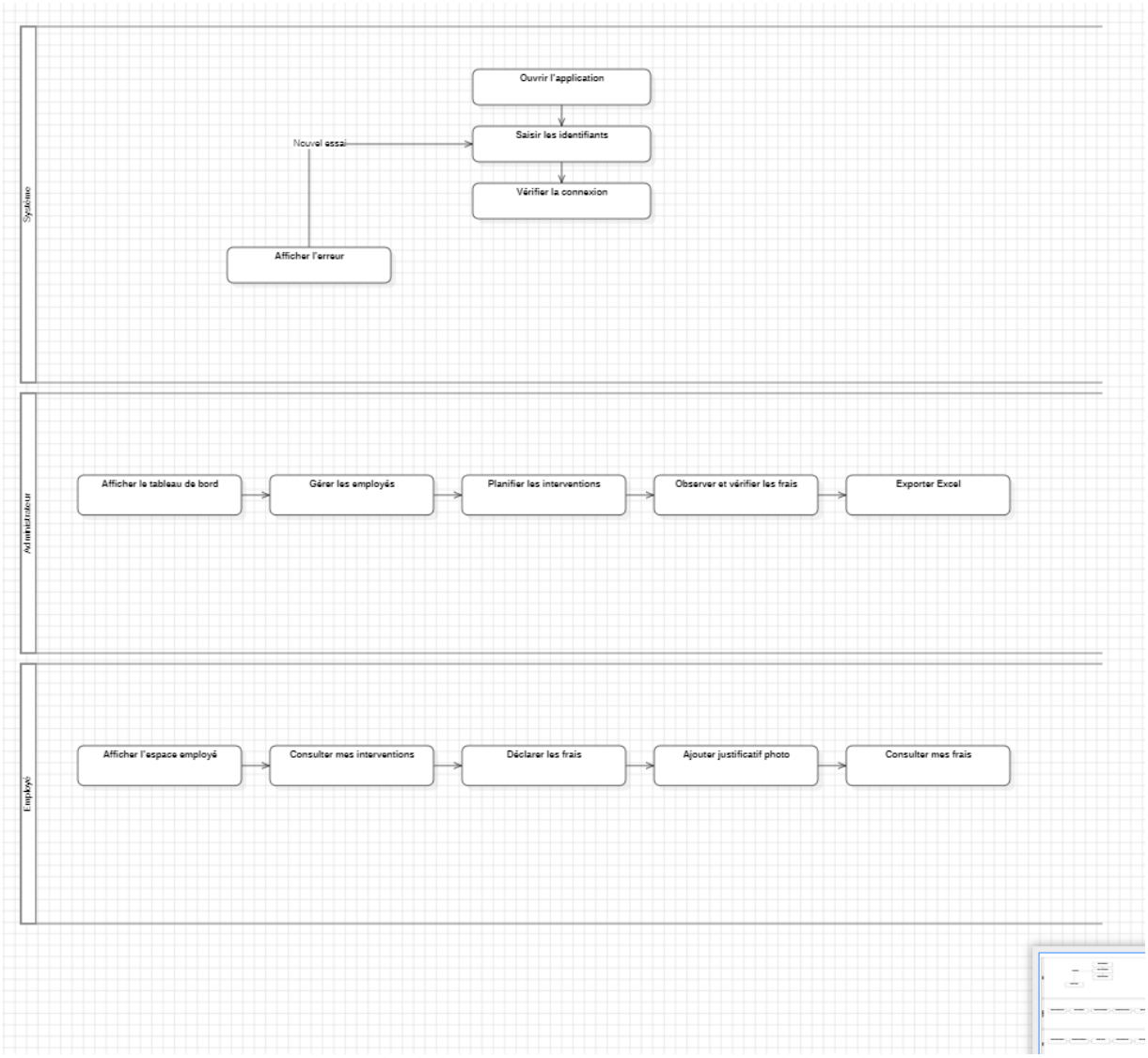


Figure 5.5 — Activités principales selon le rôle utilisateur

Après la vérification des identifiants, une erreur entraîne une nouvelle tentative de connexion. Lorsque l'authentification réussit, l'administrateur accède au tableau de bord, gère les employés, planifie les interventions, contrôle les frais et exporte les données. L'employé consulte ses interventions et déclare ses frais avec un justificatif photographique.

5.8 Conception de la sécurité

La sécurité repose sur plusieurs mécanismes complémentaires :

- hachage des mots de passe avec bcrypt via Passlib ;
- création d'un JWT signé avec l'algorithme HS256 et une durée de 60 minutes ;

- schéma OAuth2 Bearer pour extraire le jeton des requêtes ;
- dépendance require_roles appliquée aux routes sensibles ;
- contrôle de propriété pour empêcher un employé de consulter ou modifier les données d'un autre ;
- validation des données d'entrée par Pydantic avant l'accès à la base.

Dans une mise en production, la clé de signature doit être déplacée vers une variable d'environnement sécurisée. La route de création de compte administrateur doit également être protégée par un processus d'administration, et les échanges doivent être assurés par HTTPS.

5.8.1 Analyse simplifiée des menaces

Tableau 5.3 — Menaces principales et mesures prévues

Menace	Exemple dans le projet	Mesure
Usurpation d'identité	Utilisation d'un mot de passe compromis	Hachage bcrypt, durée limitée du JWT et HTTPS à prévoir.
Élévation de privilèges	Un employé appelle une route d'administration	Dépendance de rôle vérifiée côté serveur.
Accès horizontal	Un employé change l'identifiant dans l'URL	Comparaison avec employee_id extrait du compte.
Altération des données	Modification d'un frais déjà vérifié	Réinitialisation du statut lors d'une modification.
Injection	Valeur envoyée dans un formulaire	Validation Pydantic et requêtes construites par SQLAlchemy.
Divulgation de justificatif	Lecture d'une photo appartenant à un autre employé	Autorisation avant retour de la donnée ; stockage dédié recommandé.
Secret exposé	Clé JWT conservée dans le code	Externalisation obligatoire avant déploiement.

5.9 Choix de SQLite pour la version livrée

SQLite a été retenu pour disposer d'un fichier de démonstration autonome, sans serveur supplémentaire à installer. Ce choix réduit le temps de mise en route et facilite la remise du projet. Il reste cohérent avec un nombre limité d'utilisateurs pendant le stage. Il ne signifie pas que SQLite est la cible définitive : les écritures concurrentes, la sauvegarde centralisée et l'administration deviennent plus exigeantes dès que l'usage s'étend.

L'emploi de SQLAlchemy limite l'impact d'une future migration, car les accès ne reposent pas sur des requêtes SQLite dispersées dans l'application. Néanmoins, le passage à PostgreSQL nécessitera des tests, des migrations versionnées et une vérification des types, des contraintes et des données existantes.

5.10 Décisions d'interface

Les deux espaces partagent une barre latérale, une zone de titre et un bloc utilisateur. Ce choix réduit l'apprentissage lorsqu'une personne change de page. Les formulaires sont placés à côté des listes lorsque la largeur le permet : l'administrateur peut ainsi saisir puis contrôler immédiatement le résultat. Les couleurs bleu clair et blanc reprennent l'identité visuelle de l'entreprise sans surcharger les données.

Le calendrier a été privilégié pour la lecture mensuelle, tandis que les tableaux restent

plus adaptés aux montants et aux statuts. Une couleur ou un badge seul ne suffit cependant pas à transmettre un état ; les libellés textuels « à vérifier » et « vérifiée » sont donc conservés.

5.11 Conclusion du chapitre

La conception traduit les besoins en composants indépendants, en entités reliées et en parcours de validation. Cette base facilite la réalisation progressive des fonctions présentée dans le chapitre suivant.

Chapitre 6

Ce chapitre décrit les fonctionnalités développées pour les deux profils de l'application. L'administrateur peut gérer les employés, planifier les interventions, contrôler les frais et exporter les données, tandis que l'employé peut consulter ses missions et déclarer ses dépenses avec un justificatif. Il présente également la communication entre le frontend, l'API et la base de données.

Réalisation de l'application

6.1 Environnement et technologies

Les technologies ont été sélectionnées pour construire rapidement une application interne structurée, avec une séparation nette entre interface et services.

Tableau 6.1 — Technologies utilisées dans la version finale

Couche	Technologie	Utilisation
Interface	React 19, React Router	Composants, pages, états et navigation selon le rôle.
Outillage web	Vite 8	Serveur de développement et génération du bundle de production.
Communication	Axios	Appels HTTP et ajout automatique du jeton JWT.
Export	ExcelJS	Génération de la fiche journalière au format Excel.
API	FastAPI, Uvicorn	Routes REST, dépendances de sécurité et documentation OpenAPI.
Validation	Pydantic	Schémas d'entrée/sortie et règles de dates.
Persistance	SQLAlchemy 2	Modèles relationnels, sessions et requêtes.
Base	SQLite	Stockage local de la version livrée.
Sécurité	JWT, Passlib, bcrypt	Jetons d'accès et hachage des mots de passe.
Conteneurs	Dockerfiles	Images séparées pour le frontend et le backend.

6.2 Écart entre la cible initiale et la version livrée

Le document de cadrage prévoyait PostgreSQL, Docker Compose, des migrations et un dépôt GitHub structuré. La version finale transmise utilise SQLAlchemy avec SQLite et fournit deux Dockerfiles, mais pas d'orchestration Compose complète ni de scripts Alembic. Cet écart ne doit pas être masqué : le socle fonctionnel a été privilégié pendant la période du stage. Le choix de SQLAlchemy conserve toutefois une voie de migration vers PostgreSQL.

La gestion de présence, des congés et des absences figurait également dans l'idée initiale. Les sources livrées contiennent une ancienne table `leaves`, mais aucun parcours frontend/backend final ne l'expose. Cette

fonction n'est donc pas présentée comme réalisée. Elle apparaît dans les perspectives avec les règles et écrans qu'elle demanderait.

6.3 Réalisation du backend

6.3.1 Validation des interventions

La cohérence d'une période est contrôlée au niveau du schéma Pydantic. Cette position est utile, car la même règle s'applique quelle que soit la page ou le client qui appelle l'API.

Listing 6.1: *Validation de la période d'une intervention*

```
@model_validator(mode="after")
def validate_dates(self):
 if self.end_date < self.start_date:
                raise ValueError(
                        "end_date must be greater than start_date"  or  equal  to
                )
        return self
```

6.3.2 Contrôle de propriété d'un frais

La création d'un frais commence par vérifier que le compte employé possède bien une fiche liée. L'intervention demandée est ensuite chargée et son employee_id est comparé à celui de l'utilisateur courant. Ce contrôle protège l'accès même si un identifiant a été modifié manuellement dans la requête.

Listing 6.2: *Principe de contrôle avant la création d'un frais*

```
if current_user.employee_id is None:
        raise HTTPException(status_code=403)

intervention = db.query(Intervention).filter(
        Intervention.id == expense.intervention_id
).first()

if not intervention:
        raise HTTPException(status_code=404)

if intervention.employee_id != current_user.employee_id:
        raise HTTPException(status_code=403)
```

6.3.3 Calcul du plafond

Le plafond est contrôlé sur le cumul des dépenses de l'employé. Lors d'une modification, la ligne actuelle est retirée du calcul avant d'ajouter les nouvelles valeurs ; sinon le même frais serait compté deux fois. Les trois catégories sont additionnées de manière explicite. Cette règle reste simple, mais elle répond directement au besoin de limiter les montants déclarés.

6.3.4 Authentification et rôles

Les mots de passe sont hachés avec bcrypt. Après vérification, le serveur produit un JWT signé avec HS256 et une expiration de 60 minutes. Les fonctions protégées déclarent les rôles autorisés comme dépendances FastAPI. Cette manière de faire rend le contrôle visible dès la signature d'une route et limite les oublis.

Listing 6.3: *Dépendance de contrôle des rôles*

```
def require_roles(*allowed_roles):
    def role_checker(current_user=Depends(get_current_user)):
        if current_user.role not in allowed_roles:
            raise HTTPException(status_code=403)
        return current_user
    return role_checker
```

6.4 Authentification

L'écran de connexion constitue le point d'entrée unique. Il présente l'identité visuelle de CAIDELEC et un formulaire réduit aux informations nécessaires. Après une authentification réussie, le frontend stocke le jeton d'accès et détermine le rôle pour construire la navigation.



Figure 6.1 — Écran de connexion à l'application

Lors des premiers écrans, la connexion servait seulement à obtenir le jeton. Il a ensuite fallu conserver le profil et appliquer la même décision de rôle au routeur, au menu et aux appels API. La solution finale centralise les chemins autorisés, mais la sécurité réelle reste volontairement redondante : l'interface guide l'utilisateur et le backend confirme le droit.

6.5 Espace administrateur

6.5.1 Tableau de bord et calendrier

Le tableau de bord administrateur synthétise le nombre d'employés, le nombre d'interventions et le montant total des frais. Le calendrier mensuel permet d'accéder rapidement à la planification. Des raccourcis donnent accès au planning, à l'équipe et aux frais.

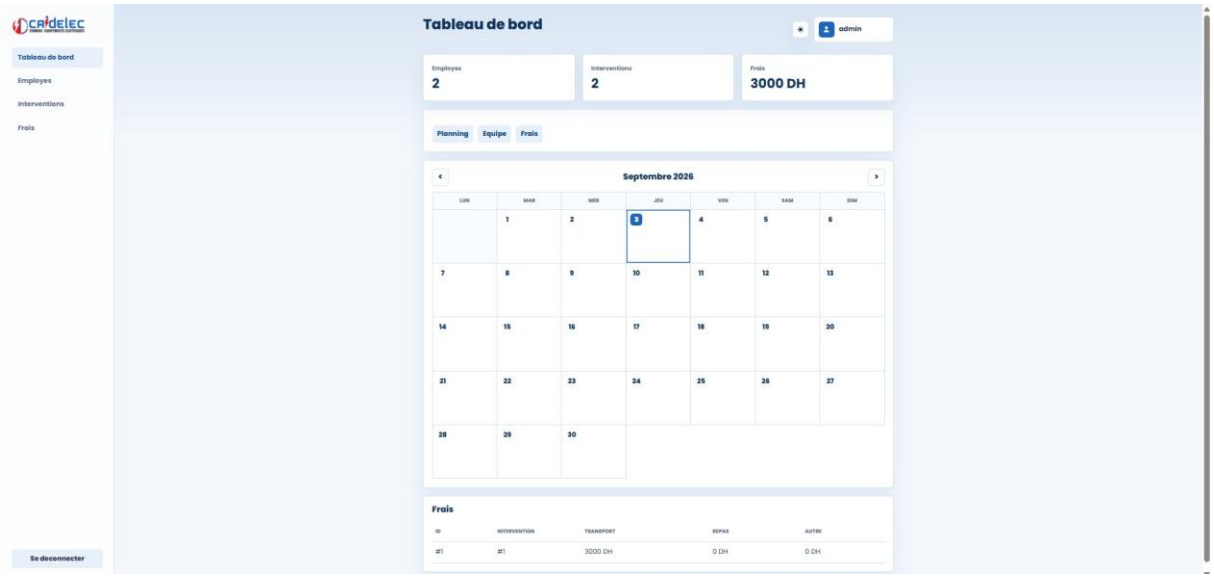


Figure 6.2 — Tableau de bord administrateur

Le calendrier est construit pour le mois affiché et associe chaque date aux interventions dont la période couvre la journée. Cette lecture est différente d'une simple liste triée : une mission de plusieurs jours doit rester visible du début à la fin. Le clic sur une date ouvre la vue détaillée utilisée pour l'export.

6.5.2 Gestion des employés

La page des employés regroupe le formulaire de création et la liste existante. Le compte utilisateur peut être créé avec la fiche, ce qui évite une opération supplémentaire et maintient le lien entre le compte et la personne. L'administrateur peut également modifier ou supprimer un employé.

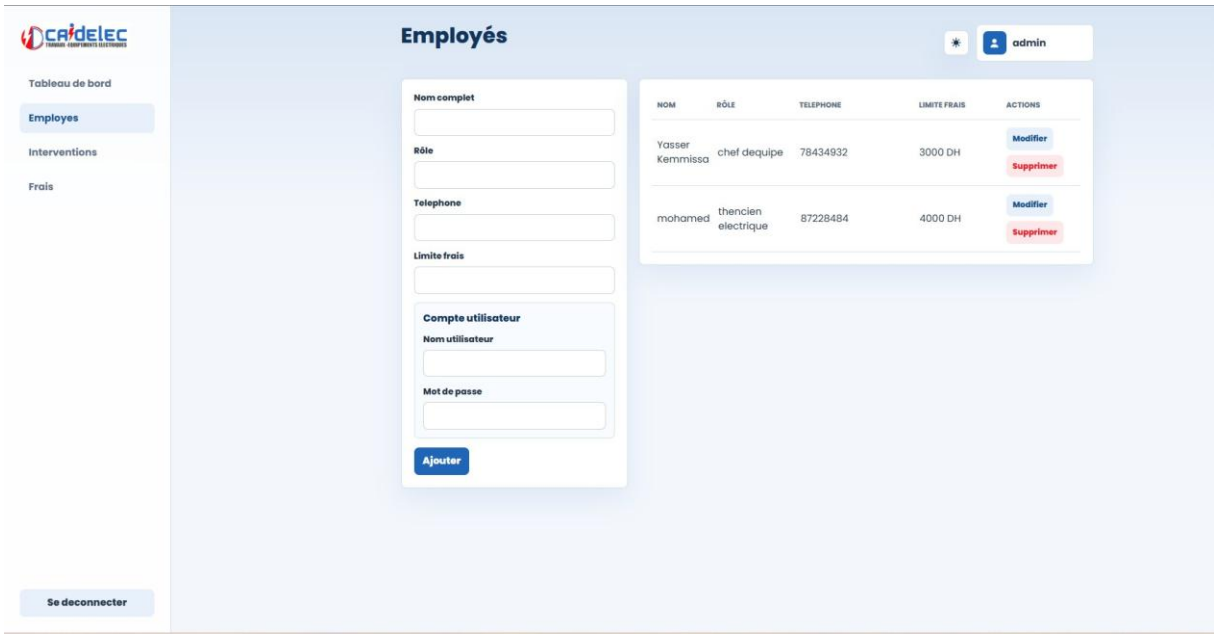


Figure 6.3 — Création et gestion des employés

Le formulaire réunit les informations professionnelles et celles du compte pour éviter une création en deux temps. Cette simplification demande cependant une transaction cohérente : si la création du compte échoue à cause d'un nom déjà utilisé, la fiche employé ne doit pas rester isolée. Pour une évolution, une unité de travail explicite et des messages de duplication en français amélioreraient encore ce parcours.

6.5.3 Planification des interventions

Le formulaire de planification contient l'employé, le chantier, la nature des travaux, une observation et la période. La liste adjacente facilite le contrôle des affectations déjà créées. L'API vérifie l'existence de l'employé et la cohérence des dates avant l'enregistrement.

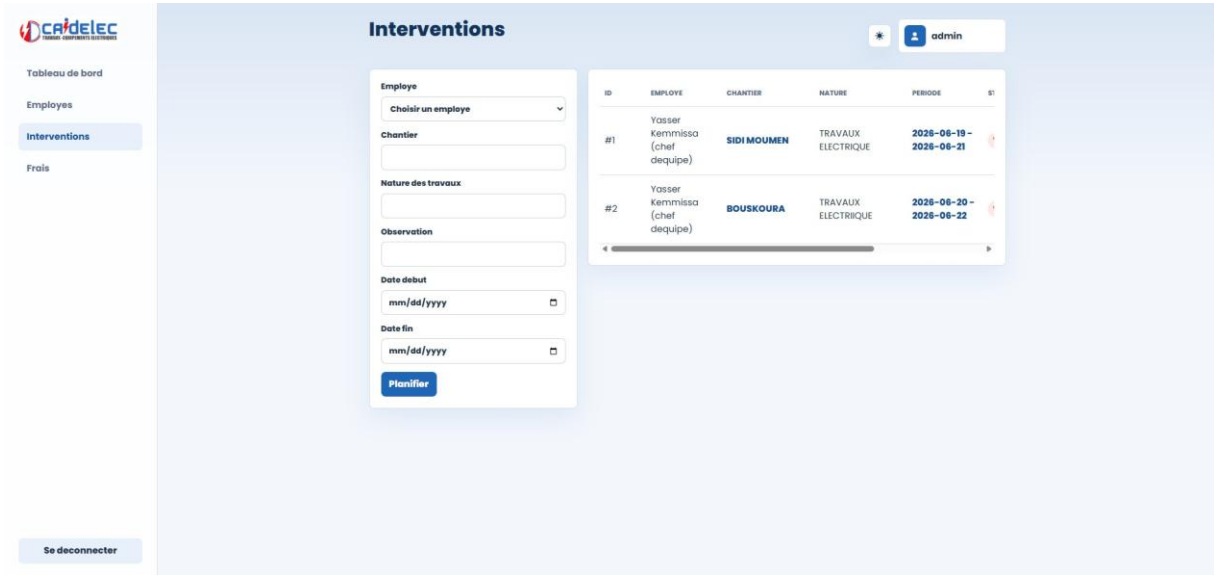


Figure 6.4 — Écran de planification des interventions

La capture fait apparaître une barre de défilement horizontale dans le tableau. Elle permet de conserver toutes les colonnes lorsque l'écran est plus étroit, mais elle n'est pas idéale pour une utilisation mobile. Une prochaine version pourrait réduire les colonnes secondaires et ouvrir le détail dans un panneau, plutôt que de forcer le défilement.

6.5.4 Contrôle des frais

La vue administrateur totalise les frais et affiche, pour chaque ligne, l'intervention, la répartition des montants, le justificatif et le statut. Le bouton de vérification n'aboutit que si un justificatif est disponible, conformément à la règle de gestion.

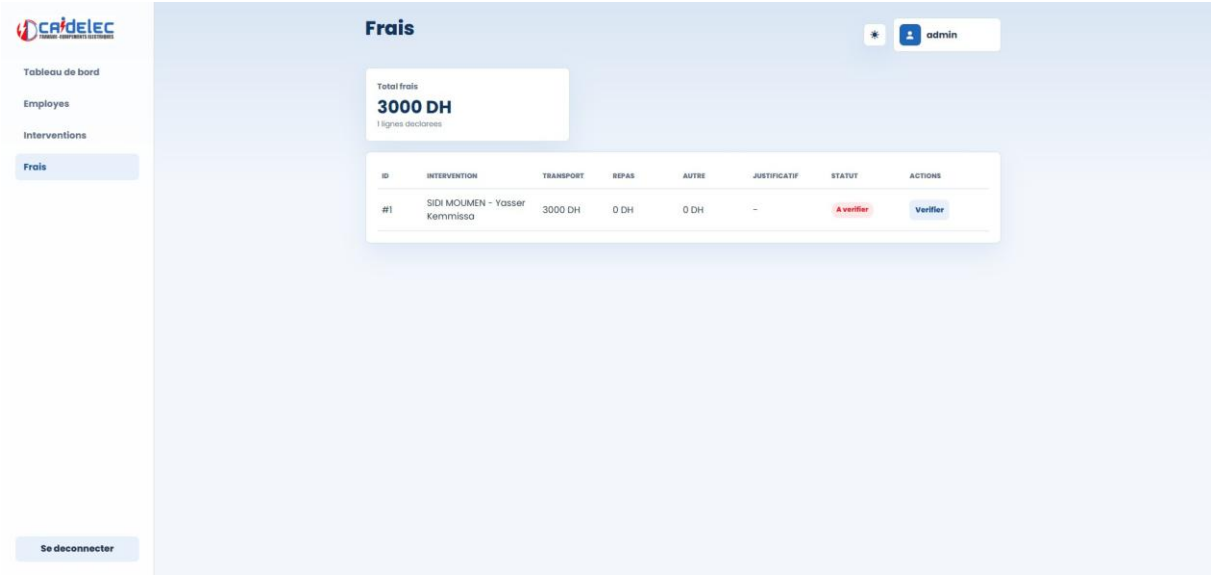


Figure 6.5 — Suivi des frais par l'administrateur

L'action « Vérifier » reste désactivée lorsque la pièce manque ou lorsque la ligne est déjà validée. Cette indication améliore l'ergonomie, mais elle ne remplace pas la règle backend : la route de vérification refuse elle aussi un frais sans receipt_photo.

6.6 Espace employé

6.6.1 Tableau de bord personnel

L'employé accède à un tableau de bord filtré : seules ses interventions et ses dépenses sont prises en compte. La structure visuelle reste proche de l'espace administrateur afin de conserver une expérience cohérente, mais les fonctions d'administration sont absentes.

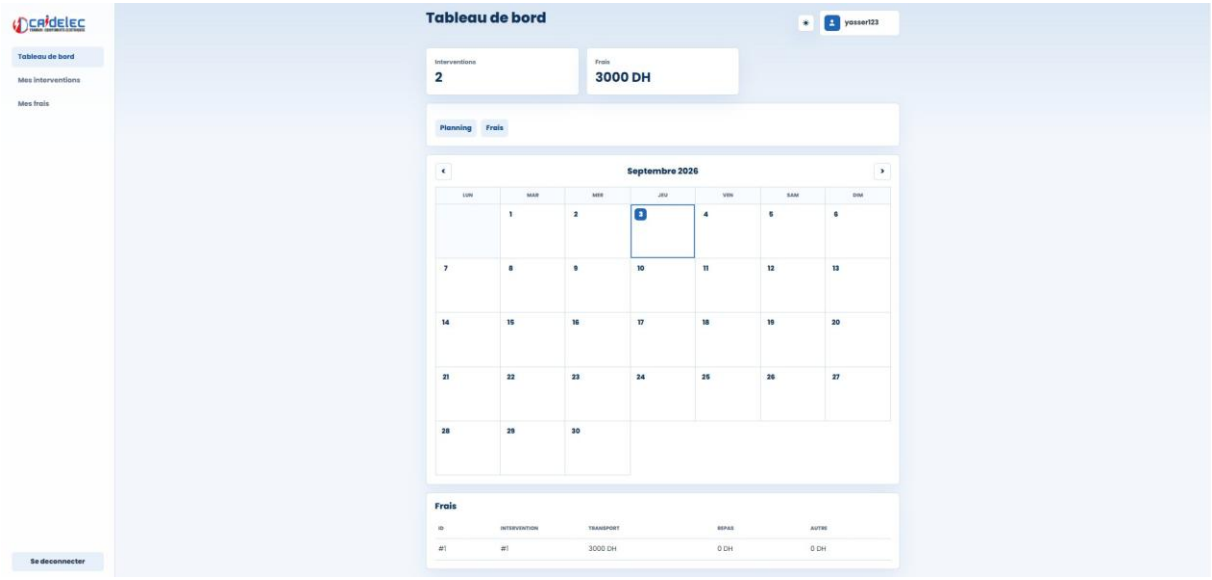


Figure 6.6 — Tableau de bord de l'employé

6.6.2 Consultation des missions

La liste « Mes interventions » affiche le chantier, la nature, la période et le statut. Un employé ne peut pas obtenir les interventions d'un autre compte : le filtrage est appliqué par l'API à partir de l'identité portée par le jeton.

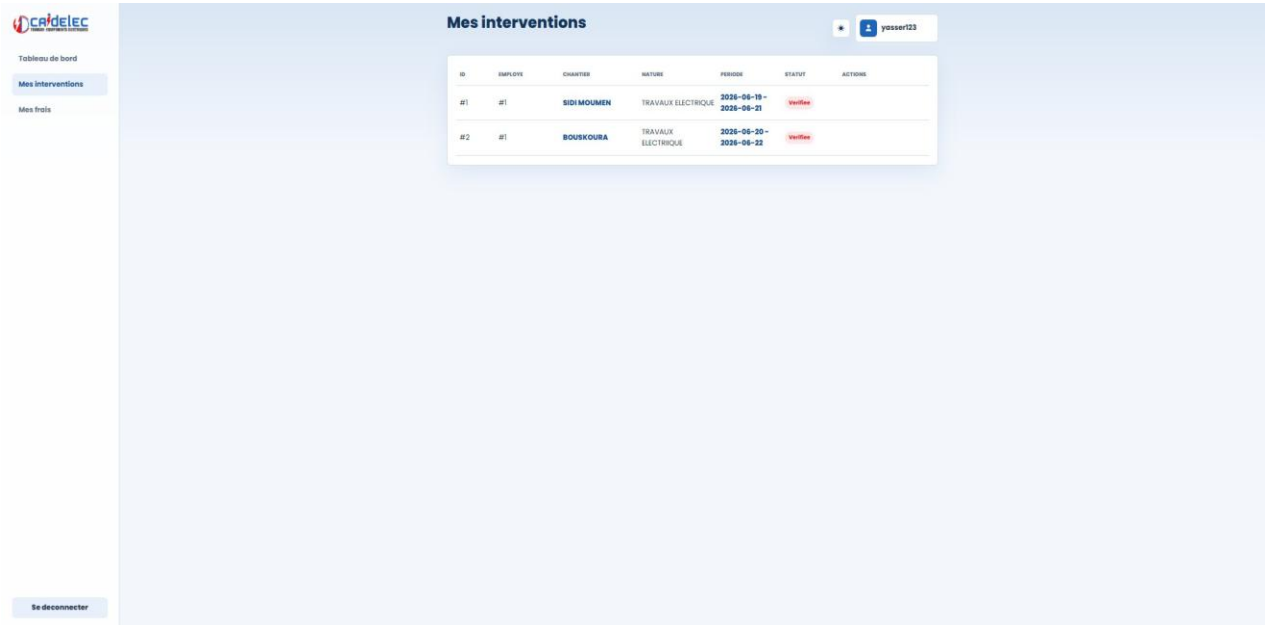


Figure 6.7 — Consultation des interventions de l'employé

6.6.3 Déclaration des frais

L'employé choisit l'une de ses interventions, répartit le montant entre transport, repas et autres dépenses, puis ajoute une photo justificative. La liste des dépenses et le total sont visibles sur la même page, ce qui permet de suivre le plafond attribué.

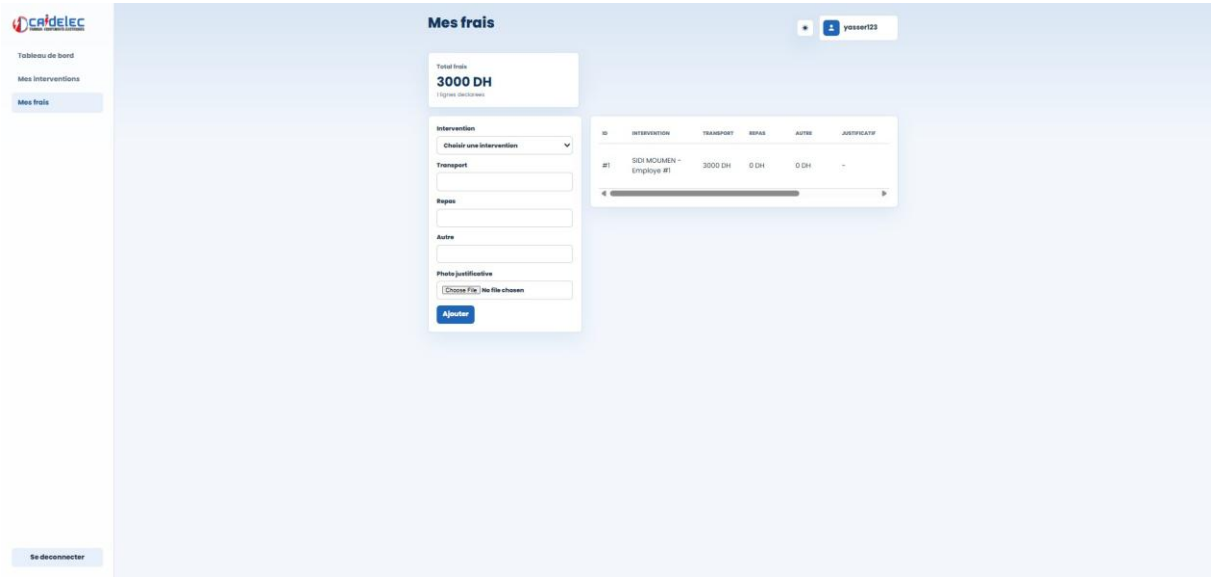


Figure 6.8 — Déclaration et suivi des frais par l'employé

La photographie choisie est lue par le navigateur et conservée dans une chaîne utilisable par l'application. Cette approche a rendu la démonstration autonome, sans service de fichiers supplémentaire. Elle augmente en revanche la taille des données et n'est pas adaptée à un grand nombre de justificatifs. Un stockage objet avec URL privée serait plus robuste en production.

6.7 Communication entre React et FastAPI

Une instance Axios contient l'adresse de base de l'API. Les pages construisent l'en-tête Bearer à partir du jeton courant. Les réponses servent directement à mettre à jour les états React et à recalculer les totaux affichés. En cas d'erreur, le message retourné par FastAPI est transformé en retour visible pour l'utilisateur.

Tableau 6.2 — Exemples d'échanges entre l'interface et l'API

Action	Requête	Mise à jour de l'écran
Connexion	POST /auth/login	Stockage du jeton puis redirection selon le rôle.
Chargement du planning	GET /interventions	Remplissage de la liste et du calendrier.
Création d'un employé	POST /employees	Réinitialisation du formulaire et rechargement de l'équipe.
Déclaration d'un frais	POST /expenses	Recalcul du total et ajout de la ligne en attente.
Vérification	PATCH /expenses/{id}/verify	Actualisation du badge de statut.

6.8 API REST

FastAPI génère automatiquement une spécification OpenAPI à partir des routes et des schémas. L'organisation par ressources rend les points de terminaison prévisibles. Les codes HTTP 400, 401, 403 et 404 signalent respectivement une donnée invalide, une authentification absente ou incorrecte, un droit insuffisant et une ressource inexistante.

Tableau 6.3 — Synthèse des ressources de l'API

Préfixe	Opérations	Contrôle principal
/auth	inscription, connexion, profil	Identité et génération du jeton
/employees	CRUD des employés	Administrateur uniquement
/interventions	CRUD, déclaration, vérification	Rôle et propriété de l'intervention
/expenses	CRUD, vérification	Rôle, propriété, plafond, justificatif

6.9 Export Excel

La vue journalière utilise ExcelJS pour générer un classeur structuré. Le document comporte le logo, la date, les chantiers, la nature des travaux, les intervenants, les contacts, les observations et des zones de signature. Cette génération automatique remplace la préparation manuelle d'une fiche quotidienne et garantit une mise en forme homogène.

La bibliothèque est chargée dynamiquement uniquement lorsque l'utilisateur demande l'export. Cette décision évite de charger tout ExcelJS pendant l'ouverture normale du tableau de bord. Le classeur est construit dans le navigateur, puis transformé en fichier téléchargeable. Aucun fichier temporaire n'est donc requis sur le serveur pour cette opération.

6.10 Difficultés rencontrées et solutions

6.10.1 Faire correspondre le compte et l'employé

Une première ambiguïté concernait le mot « rôle ». Il désigne à la fois le droit technique admin/employe et la fonction professionnelle, par exemple chef d'équipe ou technicien électrique. Les deux notions ont été séparées : le compte porte l'autorisation, tandis que la fiche employé conserve la fonction affichée.

6.10.2 Conserver un filtrage cohérent

Afficher uniquement « Mes interventions » côté React ne suffisait pas. J'ai dû appliquer le même principe dans les requêtes SQLAlchemy à partir de l'utilisateur courant. Cette double lecture a été conservée : le

frontend ne demande pas inutilement des données globales et l'API reste protégée contre une requête construite en dehors de l'interface.

6.10.3 Gérer les statuts après modification

Une dépense déjà vérifiée ne peut pas garder ce statut si son montant ou son justificatif change. La mise à jour remet donc la ligne « à vérifier ». Cette petite règle a un effet important sur la traçabilité, car elle évite de présenter comme validée une information que le responsable n'a pas revue.

6.10.4 Stabiliser les dépendances frontend

Le fichier de verrouillage npm livré n'était pas complètement aligné avec package.json. L'installation stricte par npm ci ne pouvait donc pas être utilisée telle quelle. Une installation npm install a permis de resynchroniser les métadonnées pour les vérifications locales. La bonne pratique consiste maintenant à conserver ce verrouillage à jour dans le dépôt afin que tous les environnements utilisent les mêmes versions.

6.11 Conteneurisation et exécution

Deux Dockerfiles sont fournis : le frontend s'appuie sur une image Node Alpine et lance

Vite, tandis que le backend utilise une image Python légère et démarre Uvicorn. Dans la version analysée, l'orchestration complète par Docker Compose n'est pas incluse ; elle constitue une amélioration recommandée pour exécuter l'ensemble avec une configuration unique.

6.12 Conclusion du chapitre

La réalisation couvre le cycle complet identifié pendant l'analyse : administration des personnes, planification, consultation individuelle, déclaration, contrôle et export. Les écrans témoignent d'une cohérence visuelle et d'une séparation fonctionnelle entre les deux rôles.

Chapitre 7

Ce chapitre présente les tests et les vérifications réalisés pour évaluer le fonctionnement de l'application. Les contrôles concernent notamment l'accès aux données, le plafond des frais, les justificatifs et les droits des utilisateurs. Les résultats montrent que les fonctionnalités principales sont opérationnelles, malgré certaines améliorations techniques prévues dans les prochaines versions.

Chapitre 7

Validation et résultats

7.1 Stratégie de validation

La validation a porté sur trois niveaux : la cohérence statique du backend, la construction du frontend et l'examen des scénarios fonctionnels visibles dans l'application. Il est important de distinguer les vérifications réellement exécutées des scénarios couverts par le code et les captures. La compilation Python, le build Vite et ESLint ont été exécutés dans l'environnement de vérification. La recette métier présentée ci-dessous s'appuie sur la version livrée, ses données de démonstration, les écrans fournis et la lecture des contrôles de l'API ; elle ne remplace pas une suite automatisée de tests de bout en bout.

7.2 Environnement de contrôle

Le frontend a été contrôlé avec les dépendances déclarées dans package.json. Le backend a été analysé comme un projet Python et la base SQLite livrée a été utilisée pour confirmer la présence des tables principales. L'environnement disponible ne contenait pas initialement l'ensemble des bibliothèques Python nécessaires à l'exécution complète de FastAPI ; je n'ai donc pas présenté une exécution intégrale de l'API comme un test automatisé réussi.

Cette précision rend les résultats reproductibles. Une équipe qui reprend le projet doit installer requirements.txt, démarrer Uvicorn et exécuter les scénarios HTTP avec une base temporaire avant un déploiement.

7.3 Vérifications techniques

Le backend a été analysé par compilation de l'ensemble des modules Python avec compileall. Aucun défaut de syntaxe n'a été détecté. Pour le frontend, les dépendances ont été installées dans un environnement de vérification, puis les commandes de construction et de contrôle ESLint ont été exécutées.

Tableau 7.1 — Résultats des vérifications exécutées

Vérification	Commande / méthode	Résultat
Syntaxe du backend	python -m compileall app	Succès, modules Python compilables.
Installation frontend	npm install	Succès après resynchronisation locale du fichier de verrouillage.

Vérification	Commande / méth-ode	Résultat
Bundle de production	npm run build	Succès, 88 modules transformés et bundle généré.
Qualité statique React	npm run lint	Succès, aucune erreur ESLint signalée.
Taille du bundle	Rapport Vite	Avertissement sur le bloc ExcelJS, candidat au chargement différé.

Le fichier de verrouillage npm fourni n'était pas entièrement synchronisé avec la liste des dépendances, ce qui empêchait une installation stricte par npm ci. Une exécution de npm install a recréé les métadonnées manquantes dans l'environnement de test. Pour un processus d'intégration continue reproductible, il faudra enregistrer le fichier de verrouillage mis à jour dans le dépôt.

7.4 Scénarios fonctionnels

Tableau 7.2 — Scénarios de recette fonctionnelle

N°	Scénario	Résultat attendu	État
T1	Connexion avec un compte administrateur valide	Création du jeton et ouverture du tableau de bord adminis-trateur	Observé sur l'interface
T2	Connexion avec un mot de passe incorrect	Refus avec un message d'identifiants invalides	Couvert par l'API
T3	Création d'un employé	Création de la fiche et d'un compte employé lié	Observé / donnée s livrées
T4	Planification avec date de fin antérieure	Rejet de la requête par la validation Pydantic	Couvert par le schéma
T5	Consultation des interventions par un employé	Retour limité aux interventions de l'employé connecté	Observé sur l'interface

N°	Scénario	Résultat attendu	État
T6	Déclaration d'une intervention appartenant à un autre employé	Refus HTTP 403	Couvert par l'API

N°	Scénario	Résultat attendu	État
T7	Ajout de frais dépassant le plafond	Refus et indication de la limite autorisée	Couvert par l'API
T8	Vérification d'un frais sans justificatif	Refus de la validation	Couvert par l'API
T9	Vérification après ajout d'un justificatif	Passage du statut à « vérifié »	Couvert par l'API
T10	Génération d'une fiche journalière	Téléchargement d'un classeur Excel structuré	Fonction présente

7.5 Fiches de test détaillées

7.5.1 Test de séparation des données employé

Objectif	Vérifier qu'un compte employé ne reçoit pas les interventions d'une autre personne.
Préparation	Disposer de deux fiches employé, de deux comptes liés et d'au moins une intervention par fiche.
Étapes	Se connecter avec le premier compte ; appeler la liste ; tenter ensuite de demander directement l'identifiant de l'intervention du second.
Attendu	La liste contient uniquement les lignes du premier employé et la demande directe non autorisée retourne 403.
Contrôle dans le code	Les requêtes sont filtrées par current_user.employee_id.

7.5.2 Test du plafond de frais

Objectif	Empêcher une nouvelle déclaration lorsque le cumul dépasse la limite.
Préparation	Compte employé avec une limite de 3,000 DH et dépenses existantes totalisant 2,800 DH.
Étapes	Envoyer une dépense de 150 DH, puis une autre de 100 DH ; modifier ensuite la première sans la compter deux fois.
Attendu	La première création est acceptée ; la seconde est refusée ; la modification calcule le cumul hors ligne courante.
Contrôle dans le code	Fonction ensure_expense_within_limit et paramètre exclude_expense_id.

7.5.3 Test de validation avec justificatif

Objectif	Ne pas vérifier un frais dépourvu de preuve.
Préparation	Frais au statut « à vérifier », d'abord sans photographie.
Étapes	Demander la vérification ; ajouter un justificatif ; demander à nouveau la vérification avec un compte administrateur.
Attendu	Première demande refusée avec un code 400 ; seconde acceptée et statut passé à « vérifié ».
Contrôle dans le code	La route /expenses/{id}/verify vérifie receipt_photo.

7.6 Traçabilité entre exigences et réalisation

Tableau 7.3 — Matrice de traçabilité

ID	Élément réalisé	Preuve ou contrôle	État
F01	Page Login et route /auth/login	Capture de connexion et création JWT	Réalisé
F02	Routes protégées et menus par rôle	require_roles, ProtectedRoute	Réalisé
F03	Page Employés et routeur dédié	Formulaire et tableau administrateur	Réalisé

ID	Élément réalisé	Preuve ou contrôle	État
F04	Lien employee_id dans le compte	Modèle et création combinée	Réalisé
F05	Formulaire Interventions	Validation des dates et référence employé	Réalisé
F06	Liste d'administration	Capture de planification	Réalisé
F07	Page Mes interventions	Filtre serveur par utilisateur	Réalisé
F08	Route declare-done	Changement de statut prévu	Réalisé
F09	Formulaire Mes frais	Trois catégories de montant	Réalisé
F10	Champ photo justificative	Présent dans le formulaire et le modèle	Réalisé
F11	Fonction de calcul du cumul	Refus HTTP lorsque la limite est dépassée	Réalisé
F12	Routes de vérification	Actions réservées à l'administrateur	Réalisé
F13	Tableaux de bord et calendrier	Captures des deux profils	Réalisé
F14	Export par ExcelJS	Fonction de la vue journalière	Réalisé
ID	**Élément réalisé**	**Preuve ou contrôle**	**État**
F15	Badges et champs de statut	Affichage dans les listes	Réalisé

7.7 Contrôle de la base livrée

La base SQLite contient les tables correspondant aux entités principales : utilisateurs, employés, interventions et frais. Les données de démonstration montrent notamment deux employés, des interventions rattachées et une dépense. Une table historique leaves est également présente, mais elle ne fait pas partie des parcours finaux de l'application. Sa présence confirme qu'une piste de gestion des absences a été envisagée, sans permettre de la déclarer fonctionnelle.

7.8 Critères de qualité restant à automatiser

La prochaine étape devrait transformer les scénarios précédents en tests exécutables. Pour

FastAPI, le client de test peut démarrer l'application sur une base SQLite temporaire et vérifier les statuts HTTP, le contenu JSON et les règles de propriété. Côté React, des tests de composants peuvent valider la présence des menus selon le rôle, tandis qu'un outil de test de navigateur peut couvrir le parcours complet de la connexion à la vérification.

Un pipeline minimal exécuterait l'installation reproductible, ESLint, le build frontend, les tests Python et les tests d'intégration à chaque modification. Il produirait également un rapport de couverture, sans chercher un pourcentage artificiel : les routes de sécurité et les changements de statut doivent rester prioritaires.

7.9 Analyse des résultats

Les résultats confirment la couverture du besoin principal. Les deux profils disposent d'un espace spécifique, les règles de propriété sont appliquées côté serveur et le cycle de vérification apporte la traçabilité attendue. Le calendrier et l'export réduisent le temps nécessaire à la lecture et à la diffusion du planning.

La validation fait aussi apparaître des améliorations techniques utiles. Le bundle ExcelJS est volumineux ; un chargement différé permettrait de ne télécharger ce module que lors de l'export. Par ailleurs, l'API gagnerait à disposer d'une suite de tests automatisés avec une base temporaire, d'un gestionnaire de migrations tel qu'Alembic et d'une configuration entièrement portée par des variables d'environnement.

7.10 Limites de la version actuelle

- La base SQLite convient à la démonstration et à un usage local, mais une base serveur sera préférable pour plusieurs utilisateurs simultanés.
- Le justificatif est porté par la donnée applicative ; un stockage de fichiers dédié serait plus adapté à un volume important.
- La clé JWT présente dans le code doit être externalisée avant le déploiement.
- La route d'inscription doit être encadrée afin d'empêcher la création libre d'un compte administrateur.
- L'historique détaillé des validations et des modifications n'est pas encore journalisé.
- La présence, les congés et les absences évoqués dans le besoin initial ne sont pas exposés par l'interface finale analysée.
- L'installation reproductible exige la mise à jour du fichier de verrouillage npm et une orchestration complète des services.

7.11 Conclusion du chapitre

La version réalisée satisfait les scénarios essentiels et présente une base technique cohérente. Les limites identifiées n'empêchent pas la démonstration fonctionnelle, mais elles définissent les priorités nécessaires avant une exploitation à plus grande échelle.

Conclusion générale et perspectives

Ce projet de fin d'études avait pour objectif de centraliser la planification quotidienne des équipes CAIDELEC et le suivi de leurs frais de déplacement. L'analyse a permis de distinguer les responsabilités de l'administrateur et de l'employé, puis de formaliser un cycle contrôlé allant de l'affectation d'une intervention à la vérification de sa réalisation et de ses dépenses.

La solution développée s'appuie sur une architecture SPA/API : React et Vite pour l'interface, FastAPI pour les services REST, SQLAlchemy pour l'accès aux données et SQLite pour le stockage de la version livrée. L'authentification JWT et les contrôles de rôle assurent la séparation des espaces. Les tableaux de bord, le calendrier, le justificatif photographique et l'export Excel complètent les opérations de base et renforcent la visibilité du responsable.

Au-delà du résultat fonctionnel, ce projet a permis de mettre en pratique l'analyse d'un besoin métier, la modélisation UML, la conception d'une API, la gestion d'état côté client, la sécurité applicative et la validation d'un livrable complet. Il illustre le rôle d'un profil GSI capable de relier une problématique organisationnelle à une réponse informatique structurée.

Les perspectives prioritaires sont les suivantes :

– migrer vers PostgreSQL pour une exploitation multiutilisateur ;

– ajouter Alembic et un processus versionné de migration des données ;

– externaliser les secrets, activer HTTPS et renforcer la gestion des comptes ;

– stocker les justificatifs dans un service de fichiers protégé ;

– mettre en place des notifications lors d'une affectation ou d'une validation ;

– produire des rapports PDF mensuels et des statistiques par chantier ou par employé ;

– ajouter un historique d'audit des changements de statut ;

– intégrer la gestion des présences, des congés et des absences ;

– automatiser les tests de l'API et du frontend dans une chaîne d'intégration continue ;

– compléter la conteneurisation par une configuration Docker Compose adaptée au déploiement.

Ainsi, l'application constitue un socle opérationnel pour la digitalisation du planning et des frais. Son architecture modulaire permet d'envisager une évolution progressive vers une plateforme interne plus large de pilotage des équipes de terrain.

Références bibliographiques

[1] FastAPI, *Documentation officielle*, disponible sur : https://fastapi.tiangolo. com/, consultée en 2026.

[2] Meta Open Source, *React Documentation*, disponible sur : https://react.dev/, consultée en 2026

[3] Vite, *Guide officiel*, disponible sur : https://vite.dev/guide/, consulté en 2026.

[4] SQLAlchemy, SQLAlchemy Documentation, disponible sur : https://docs. sqlalchemy.org/, consultée en 2026.

[5] SQLite Consortium, *SQLite Documentation*, disponible sur : https://www.sqlite. org/docs.html, consultée en 2026.

[6] Pydantic, *Pydantic Documentation*, disponible sur : https://docs.pydantic.dev/, consultée en 2026.

[7] M. Jones, J. Bradley et N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015, https://www.rfc-editor.org/rfc/rfc7519.

[8] Axios, *Documentation officielle*, disponible sur : https://axios-http.com/docs/ intro, consultée en 2026.

[9] ExcelJS, *Documentation du projet*, disponible sur : https://github.com/exceljs/ exceljs, consultée en 2026.

[10] Docker, *Docker Documentation*, disponible sur : https://docs.docker.com/, consultée en 2026.

Annexe A — Protocole de démonstration

Cette annexe décrit un parcours court permettant de vérifier devant le jury que les fonctions annoncées dans le rapport correspondent à une application exécutable. Les données utilisées pendant la démonstration doivent être des données de test et ne contenir aucun mot de passe réel.

Étape	Action	Résultat attendu	Élément observable
1	Démarrer l'API et l'interface	Les deux services répondent sans erreur	Terminal et page de connexion
2	Se connecter comme administrateur	Le tableau de bord administrateur s'affiche	Nom du compte, indicateurs et calendrier
3	Créer un employé et son compte	L'employé apparaît dans la liste	Nouvelle ligne et limite de frais
4	Planifier une intervention	L'intervention est enregistrée et visible	Chantier, période, employé et statut
5	Se connecter comme employé	Seules les données liées au compte sont visibles	Tableau de bord et liste personnelle
6	Déclarer un frais avec justificatif	La dépense est créée avec le statut « à vérifier »	Montants, photo et retour de l'interface
7	Vérifier le frais comme administrateur	Le statut devient « vérifié » ou « rejeté »	Mise à jour dans les deux espaces
8	Exporter les affectations	Un fichier Excel est généré	Fichier lisible avec la journée sélectionnée

En cas d'erreur, la démonstration doit montrer le message retourné et expliquer le contrôle concerné : authentification, autorisation par rôle, propriété de l'intervention, données obligatoires ou plafond de frais.

Annexe B — Traçabilité de l'évaluation

La matrice suivante relie les éléments annoncés dans le rapport à une preuve contrôlable pendant l'évaluation. Elle sert de support à la démonstration et ne remplace pas l'explication orale des choix réalisés.

Élément évalué	Implémentation	Preuve attendue	Justification
Séparation des rôles	JWT et contrôles FastAPI	Deux espaces distincts	Limiter les fonctions et les données selon le profil
Cohérence des affectations	Relations Employee–Intervention	Intervention rattachée à un employé	Éviter les missions sans responsable
Traçabilité des frais	Expense liée à Intervention	Montants, justificatif et statut	Relier chaque dépense à une mission
Contrôle du plafond	Validation côté API	Refus d'une dépense non conforme	Appliquer la règle même si la requête est modifiée
Pilotage	Calendrier et indicateurs	Tableau de bord actualisé	Donner une vue synthétique à l'administrateur
Export	Génération Excel	Fichier correspondant à la sélection	Faciliter la transmission du planning

Annexe C — Exécution de la version livrée

La livraison comprend un backend FastAPI, un frontend React/Vite et une base SQLite de démonstration. Le fichier requirements.txt doit être placé avec le backend avant l'installation des dépendances Python.

Composant	Commande	Résultat
Backend	python -m venv .venv	Environnement Python isolé
Backend	pip install -r requirements.txt	Dépendances installées
Backend	uvicorn app.main:app --reload	API disponible localement
Frontend	npm install	Dépendances JavaScript installées
Frontend	npm run dev	Interface Vite disponible localement
Contrôle	npm run build	Construction de production sans erreur
Contrôle	npm run lint	Analyse statique du frontend

Avant la soutenance, il faut démarrer les deux services, vérifier l'URL de l'API dans le frontend, préparer deux comptes de test et conserver une copie de la base de démonstration. Aucun secret réel ne doit apparaître dans le rapport, les captures ou la présentation.

Annexe D Corrections et fonctionnalite ajoutee

Cette annexe presente les corrections ajoutees apres la soutenance et integre les nouveaux diagrammes UML. La fonctionnalite ajoutee est le module Rapports et Statistiques. Elle permet de consulter les indicateurs par periode, de regrouper les frais par employe et par intervention, puis d'exporter les resultats au format CSV.

Ce module renforce l'application parce qu'il ne se limite pas a la saisie des donnees. Il donne une vision de pilotage aux responsables et permet a chaque employe de consulter ses propres statistiques sans acceder aux donnees des autres utilisateurs.

Tableau D.1 - Fonctionnalite supplementaire ajoutee

Element	Description
Endpoint backend	GET /reports/summary avec filtres start_date et end_date.
Interface frontend	Page Rapports pour admin et Mes rapports pour employe.
Controle d'acces	Admin consulte toutes les donnees. Employe consulte seulement ses donnees.
Indicateurs	Total frais, interventions, frais par statut, frais par employe et frais par intervention.
Export	Export CSV telechargeable depuis l'interface.

Tests de verification ajoutes

Tableau D.2 - Scenarios de verification du module rapports

Scenario	Resultat attendu
Connexion admin puis ouverture Rapports	Les statistiques globales sont affichees.
Filtrage par periode	Les interventions et frais hors periode ne sont pas inclus.
Export CSV	Le fichier contient les totaux et les regroupements principaux.
Connexion employe	Seules les donnees de l'employe connecte sont affichees.
Acces non autorise	Le backend refuse toute consultation en dehors du role autorise.

Diagramme de cas d'utilisation corrige

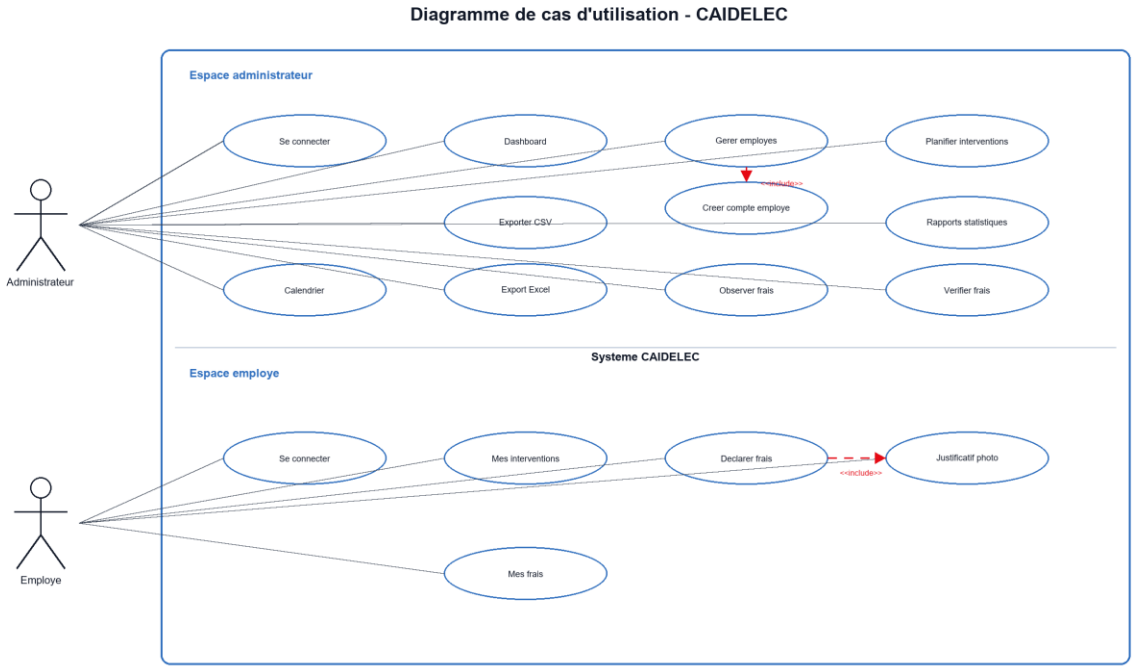


Figure D.1 - Diagramme de cas d'utilisation avec module Rapports

Diagramme de classes corrige

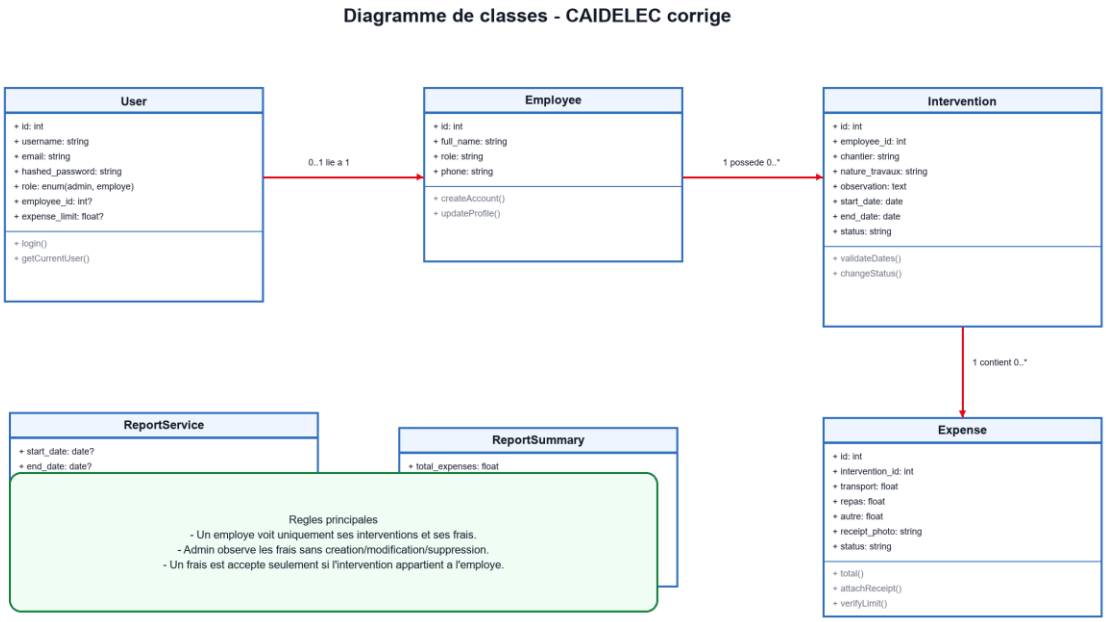


Figure D.2 - Diagramme de classes avec ReportService et ReportSummary

Diagramme de sequence corrige

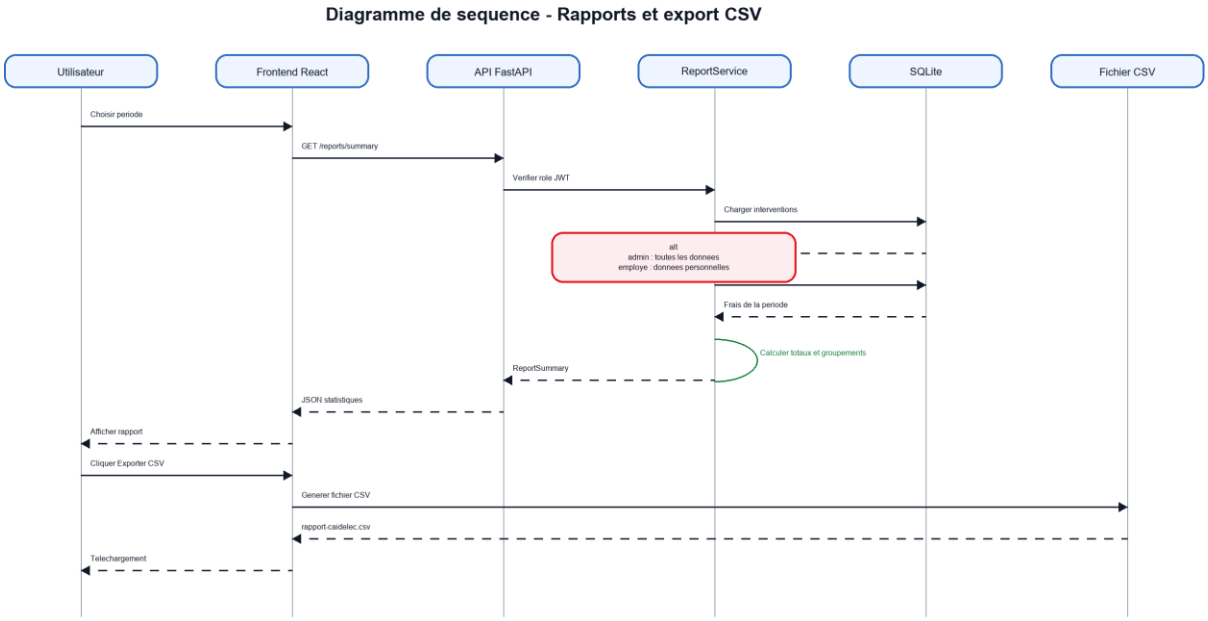


Figure D.3 - Sequence de consultation des rapports et export CSV

Diagramme d'activite corrige

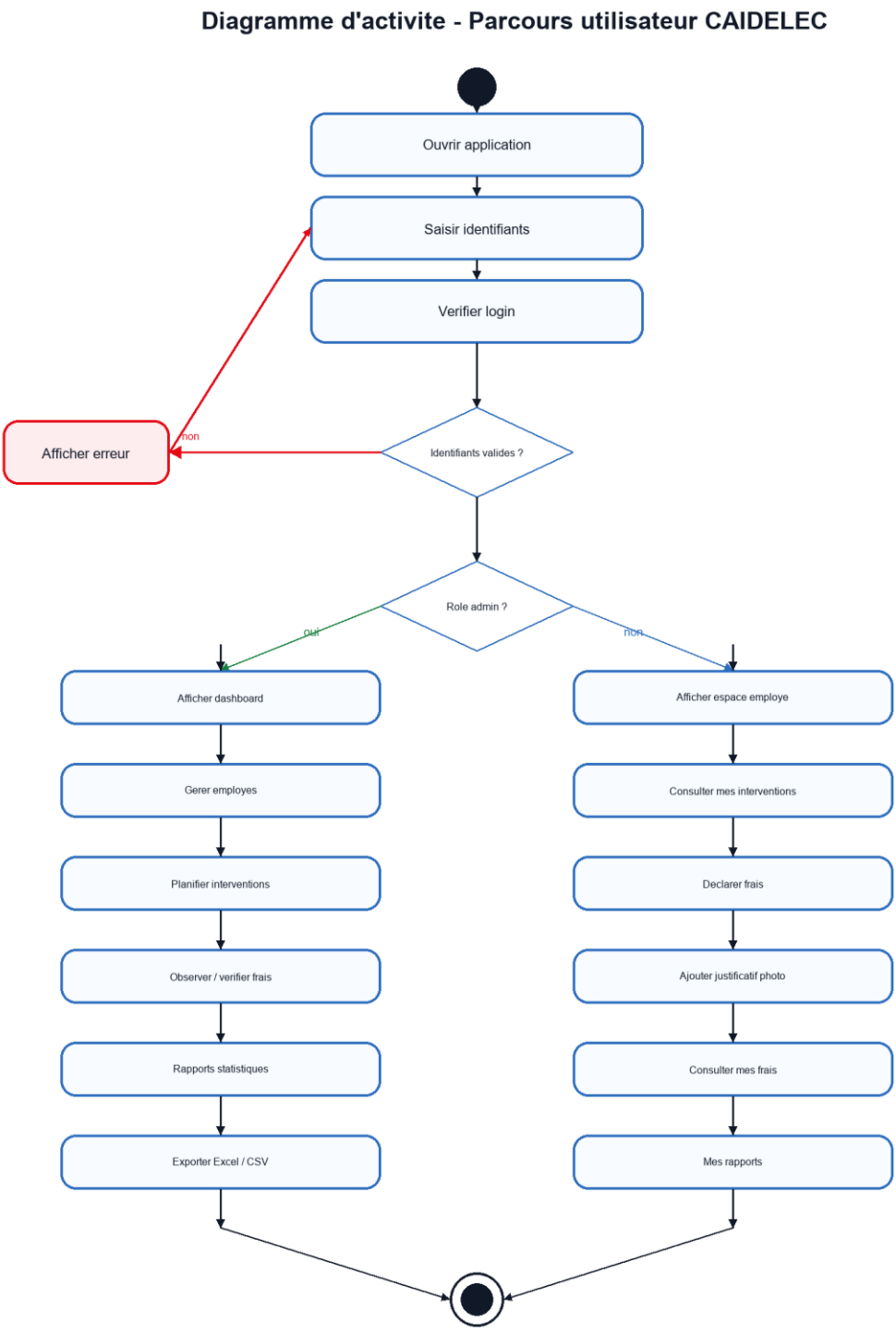


Figure D.4 - Activites principales avec rapports statistiques